NIVEAU EXPERT

MANUEL DE L'APPRENANT

Créer avec l'IA

Le vibe coding, de l'idée à l'application en ligne



Kouadio Koffi Elisée

Situations de vie · exercices · cas pratiques

FOR LIFE

COLLECTION « APPRIVOISER L'INFORMATIQUE » · TOME 3

Créer avec l'IA

Le vibe coding, de l'idée à l'application en ligne

Manuel de formation

Niveau expert

Kouadio Koffi Elisée

FOR LIFE · Abidjan

© 2026 Kouadio Koffi Elisée — FOR LIFE. Tous droits réservés.

Toute reproduction, même partielle, sans l'accord écrit de l'auteur est interdite.

Outils de référence : des plateformes de création d'applications par intelligence artificielle (Lovable, Bolt, Replit, Google AI Studio…), des éditeurs assistés par IA (Cursor, Claude Code…), GitHub, Vercel, Netlify, Supabase, Make et Zapier. Les noms et les offres changent vite : la méthode, elle, reste.

Les noms de marques cités appartiennent à leurs propriétaires respectifs. Les personnages et les situations sont fictifs ; toute ressemblance serait fortuite.

Les services en ligne évoluent vite : si un bouton a changé de nom ou de place, cherchez la fonction équivalente, la méthode reste la même.

Première édition, 2026.

Préface : du lecteur à l'auteur

Pendant longtemps, créer une application était réservé à une petite élite : des années d'études, des langages compliqués, des équipes coûteuses. Une commerçante d'Adjamé, un étudiant de Cocody ou une association de Korhogo qui avaient une idée devaient payer cher, attendre longtemps, ou renoncer.

En février 2025, l'informaticien Andrej Karpathy a donné un nom à une nouvelle manière de faire : le **vibe coding**. On décrit ce qu'on veut à une intelligence artificielle, en langage courant, et elle écrit le code. On essaie, on corrige en lui parlant, et l'application prend forme. Ce qui demandait des mois peut prendre des jours. Ce qui coûtait des millions peut coûter le prix d'un abonnement.

Ce n'est pas de la magie, et ce livre ne vous le fera pas croire. L'IA écrit vite, mais elle se trompe, elle oublie la sécurité, elle tourne en rond. Karpathy lui-même rappelait que la méthode brute convient aux petits projets du week-end. Pour livrer une vraie application à un vrai client, il faut une **méthode** : **décrire, construire, vérifier, corriger, sauvegarder, publier, protéger**. C'est cette méthode que vous allez apprendre ici.

Vous n'allez pas devenir programmeur. Vous allez devenir quelque chose de plus rare : un **maître d'ouvrage du numérique**, capable de transformer un besoin réel, de chez nous, en un outil qui fonctionne. Une vitrine pour la boutique du quartier. Une application pour la tontine. Un système de commande pour la coopérative. Et peut-être, demain, un métier.

Dans le Tome 1, vous avez appris à lire le numérique. Dans le Tome 2, à l'écrire avec aisance. Ici, vous en devenez l'auteur.

Kouadio Koffi Elisée

Fondateur de FOR LIFE, formateur

Sommaire

Préface : du lecteur à l'auteur 4
Comment utiliser ce livre 6
Test de positionnement 7
Module 1 · Le vibe coding, c'est quoi ? 8
Module 2 · Penser comme un programmeur, sans coder 14
Module 3 · Comprendre ce que l'IA fabrique 21
Module 4 · L'art de parler à l'IA pour coder 28
Module 5 · Ma première application de A à Z 34
Module 6 · Une vraie application avec comptes et données 40
Module 7 · Quand ça ne marche pas : réparer avec l'IA 47
Module 8 · Sauvegarder chaque étape avec GitHub 53
Module 9 · Mettre son application en ligne 59
Module 10 · Une application sûre et de qualité 65
Module 11 · Brancher son application au monde 71
Module 12 · Automatisation et IA avancée : API, Make, Zapier, agents 77
Module 13 · Vivre du vibe coding 83
Projet final · Une application complète pour un vrai client 90
Annexe A · Glossaire 92
Annexe B · La bibliothèque de prompts 97
Annexe C · Pour aller plus loin 101

Comment utiliser ce livre

Ce tome garde **la même structure** que le Tome 1. Il suppose acquis les Tomes 1 et 2, en particulier la sécurité des comptes, le travail en ligne et l'usage raisonné de l'intelligence artificielle. **Vous n'apprendrez pas à écrire du code** : vous apprendrez à le faire écrire, à le comprendre assez pour le diriger, et à le livrer.

Étape	Ce que vous y trouvez
📖 **La situation**	Une histoire de la vie professionnelle ou quotidienne.
💡 **Le déclic**	L'explication, toujours à partir d'une image de la vie réelle.
🖱️ **Pas à pas**	Les manipulations, étape par étape.
⚠️ **Les pièges à éviter**	Les erreurs fréquentes et les questions qu'on n'ose pas poser.
✍️ **Exercices**	🟢 *Je m'échauffe*, 🟡 *Je m'entraîne*, 🔴 *Je me lance*.
🧩 **Cas pratique**	Un vrai problème à résoudre.
📌 **Ce que je retiens**	La fiche mémo du module.
🏆 **Mon défi de la semaine**	Une mission dans votre vraie vie.

Les personnages, deux ans après leurs débuts

Qui ?	Où en est-il ?
Yao	Diplômé, il veut devenir créateur d'applications indépendant, sans avoir fait d'école d'informatique.
Koffi	Son secrétariat est devenu une petite entreprise. Il veut ses propres outils : prise de rendez-vous, suivi des clients, paiement en ligne.
Mariam	Assistante de direction, elle veut automatiser les tâches répétitives de son service.
Tantie Akissi	Première cliente de Yao : elle veut une vitrine en ligne et une application pour sa tontine.
M. Kouassi	Il a lui-même appris le vibe coding. Il en connaît la puissance… et les pièges.

> 📚 **Les logiciels et services utilisés**
>
> La plupart des plateformes proposent une offre gratuite limitée, suffisante pour apprendre ; les projets réels demandent souvent un abonnement de quelques dizaines de dollars par mois. Vérifiez les prix et les conditions au moment où vous lisez. Les exemples de ce livre ne dépendent d'aucun outil précis : chaque étape indique ce qu'il faut chercher dans l'outil que vous utilisez.

Test de positionnement : par où commencer ?

Cochez **Oui** seulement si vous savez le faire **seul, sans aide**.

Je sais…	Oui	Non
1. Expliquer ce qu'est le vibe coding et ses limites	☐	☐
2. Rédiger un petit cahier des charges : utilisateurs, écrans, actions, données	☐	☐
3. Expliquer la différence entre interface, serveur et base de données	☐	☐
4. Écrire un premier prompt complet pour créer une application	☐	☐
5. Créer une page web qui fonctionne sur téléphone avec un outil d'IA	☐	☐
6. Ajouter une inscription, une connexion et des données à une application	☐	☐
7. Lire un message d'erreur et le transmettre efficacement à l'IA	☐	☐
8. Sauvegarder un projet sur GitHub et revenir à une version précédente	☐	☐
9. Mettre une application en ligne avec un nom de domaine	☐	☐
10. Protéger les clés secrètes et les données des utilisateurs	☐	☐
11. Ajouter un paiement par Mobile Money à une application	☐	☐
12. Créer une automatisation avec Make ou Zapier	☐	☐
13. Expliquer ce qu'est un agent d'IA et comment le garder sous contrôle	☐	☐
14. Chiffrer une prestation et rédiger un devis	☐	☐

Mon résultat

Nombre de « Oui »	Mon parcours conseillé
0 à 5	Suivez tout le livre dans l'ordre. Les modules 1 à 4 posent les fondations : ne les sautez pas.
6 à 10	Survolez les modules 1 à 3, puis travaillez à partir du premier module où vous avez répondu « Non ».
11 à 14	Lancez-vous directement dans le projet final, en revenant aux modules au besoin.

MODULE 1

Le vibe coding, c'est quoi ?

Vous êtes le maître d'ouvrage, l'IA est le maçon

🎯 **À la fin de ce module, je saurai…**

- expliquer ce qu'est le **vibe coding**, d'où il vient et ce qu'il change ;
- distinguer ce que l'IA **sait faire** et ce qu'elle **ne sait pas faire** ;
- connaître les **familles d'outils** et choisir par où commencer ;
- comprendre le **cycle** de travail : décrire, construire, vérifier, corriger, publier ;
- créer ses **comptes** et préparer son poste de travail.

Durée conseillée	Matériel	Prérequis
Trois heures	Un ordinateur connecté, une adresse e-mail sérieuse	Tomes 1 et 2

📖 LA SITUATION

Les deux millions de Koffi

Koffi veut une application pour son entreprise : ses clients prendraient rendez-vous en ligne, recevraient un rappel, et lui verrait son planning sur son téléphone. Il a demandé un devis à une agence du Plateau : **millions de F**, quatre mois de délai, et 50 000 F par mois de maintenance.

Le soir, Yao lui montre son téléphone : « Grand-frère, regarde. Cette page de rendez-vous, je l'ai faite cet après-midi en parlant à une IA. Elle n'est pas finie, mais elle marche. »

Koffi : Tu me racontes quoi ? Tu ne sais pas programmer !

M. Kouassi : Il ne sait pas programmer, Koffi, c'est vrai. Mais il sait ce qu'il veut, il sait le décrire, et il sait vérifier. Pour beaucoup de projets, c'est devenu l'essentiel. Pas pour tous. Et c'est justement ce qu'il faut apprendre à distinguer.

💡 LE DÉCLIC

Le maître d'ouvrage et le maçon

Quand on construit une maison, le **maître d'ouvrage**, c'est celui qui sait ce qu'il veut : trois chambres, une cuisine ouverte, une terrasse côté lagune. Il ne pose pas les briques lui-même. Le **maçon** construit, vite et bien… à condition de recevoir des consignes claires. Et le maître d'ouvrage passe régulièrement sur le chantier pour **vérifier** : un mur de travers se corrige au début, pas à la fin.

Dans le **vibe coding**, vous êtes le maître d'ouvrage. L'intelligence artificielle est le maçon. Vous décrivez en français ce que vous voulez ; elle écrit le **code**, c'est-à-dire les instructions que comprend l'ordinateur. Vous essayez le résultat, vous demandez des corrections, et l'application avance.

> 📜 **D'où vient le mot**
>
> Le 2 février 2025, l'informaticien **Andrej Karpathy**, ancien responsable de l'intelligence artificielle chez Tesla et cofondateur d'OpenAI, a décrit une nouvelle façon de programmer où l'on « se laisse porter par les vibrations » (*vibes*) et où l'on oublie presque que le code existe. Le mot est devenu célèbre en quelques semaines. Karpathy précisait lui-même que cette manière brute convient aux petits projets « du week-end ». Ce livre vous apprend à aller plus loin, avec méthode.

Ce que le maçon IA sait faire, et ce qu'il ne sait pas faire

L'IA sait…	L'IA ne sait pas…
Écrire très vite le code d'une page, d'un formulaire, d'un tableau de bord	Deviner ce que vous voulez si vous ne le dites pas
Proposer un design propre et adapté au téléphone	Connaître vos clients, vos prix, vos règles de gestion
Corriger une erreur qu'on lui décrit bien	Garantir que tout est juste et sûr : elle peut se tromper avec assurance
Expliquer le code qu'elle a écrit	Penser seule à la sécurité, aux cas particuliers, à la loi
Relier l'application à une base de données, à un paiement, à des e-mails	Assumer la responsabilité devant le client : c'est vous

Le cycle du vibe codeur

Tout le livre suit le même cycle, que vous répéterez des centaines de fois :

Étape	Ce que vous faites	Module
1. **Décrire**	Penser le besoin, écrire le cahier des charges et le prompt	2, 4
2. **Construire**	Laisser l'IA produire, par petites étapes	5, 6
3. **Vérifier**	Essayer comme un utilisateur, sur ordinateur et téléphone	3, 10
4. **Corriger**	Décrire précisément ce qui ne va pas	7
5. **Sauvegarder**	Garder chaque version qui marche	8
6. **Publier**	Mettre en ligne, à une vraie adresse	9
7. **Protéger et relier**	Sécurité, paiement, automatisations	10, 11, 12

🖱️ PAS À PAS

Étape 1 : les familles d'outils

Famille	Exemples	Pour qui, pour quoi
Les constructeurs d'applications dans le navigateur	Lovable, Bolt, Replit, Google AI Studio, v0	Le meilleur point de départ : on décrit, l'application apparaît avec un aperçu, on publie en un clic. C'est l'outil principal de ce livre.
Les éditeurs de code assistés par IA	Cursor, Windsurf, Visual Studio Code avec un assistant	Plus de contrôle, pour des projets plus gros. Demande d'installer un logiciel et de manipuler des fichiers.
Les agents de code	Claude Code, et des agents similaires	Une IA qui travaille sur tout un projet, lit les fichiers, lance des commandes, propose des modifications. Puissants, à superviser de près.
Les services autour	GitHub (sauvegarde), Vercel et Netlify (mise en ligne), Supabase et Firebase (données et comptes), Make et Zapier (automatisation)	Ils complètent l'application. Nous les verrons module par module.

> 💡 **Le conseil de M. Kouassi**
>
> « Choisissez **un** constructeur d'applications et restez-y pendant tout ce livre. Les outils se ressemblent : quand vous en maîtrisez un, passer à un autre prend une après-midi. Ce qui compte, c'est la méthode. »

Étape 2 : préparer son poste de travail

1. Utilisez une adresse e-mail **dédiée** à vos projets (par exemple **prenom.nom.dev@gmail.com**), protégée par la double authentification (Tome 2, module 8).
2. Créez un compte sur **un** constructeur d'applications de votre choix (offre gratuite pour commencer).
3. Créez un compte **GitHub** (github.com) : il servira à sauvegarder vos projets (module 8).
4. Utilisez votre **gestionnaire de mots de passe** pour tous ces comptes.
5. Créez dans Documents un dossier **Projets** et, dans votre outil de notes, un **journal de bord** : vous y noterez vos prompts réussis, vos erreurs et leurs solutions.

Étape 3 : votre toute première création

Pour sentir la puissance de l'outil, faites ce petit essai, sans chercher la perfection :

✍️ Prompt d'essai

« Crée une page web d'une seule page pour présenter Pagnes Akissi, une boutique de pagnes au marché d'Adjamé à Abidjan. Elle doit contenir : un titre accueillant, une courte présentation, trois photos d'exemple de pagnes avec leur prix en francs CFA, les horaires d'ouverture et un gros bouton vert « Commander sur WhatsApp ». Couleurs chaudes : orange, vert, crème. La page doit être belle sur téléphone. »

1. Collez ce prompt dans votre constructeur d'applications et envoyez.
2. Regardez l'IA travailler, puis observez l'**aperçu**.
3. Ouvrez l'aperçu en mode **téléphone** (icône de téléphone, ou réduisez la fenêtre).
4. Notez dans votre journal de bord trois choses que vous aimez et trois choses que vous voudriez changer. Nous les changerons au module 5.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
🪄 Croire que l'IA fait tout, toute seule	Applications fragiles, fausses, dangereuses.	Vous restez le maître d'ouvrage : vous décidez et vous vérifiez.
🔀 Changer d'outil tous les trois jours	On n'en maîtrise aucun.	Un constructeur pour tout le livre.
🏗️ Promettre tout de suite une grosse application à un client	Délais intenables, client déçu.	Commencer par de petits projets maîtrisés.
💸 Ignorer les coûts des abonnements et des crédits	Factures surprises en fin de mois.	Lire les offres, surveiller sa consommation.
🙈 Ne jamais regarder le code	On ne comprend rien quand ça casse.	Le module 3 vous apprend à le lire sans l'écrire.

🤦🏾 Les questions qu'on n'ose pas poser

« Faut-il quand même apprendre à programmer ? »

Pour commencer, non. Mais plus vous comprendrez comment une application est construite, mieux vous dirigerez l'IA. Ce livre vous donne juste ce qu'il faut ; si le métier vous plaît, apprendre les bases de la programmation sera un excellent investissement.

> **« Les applications faites par IA sont-elles de vraies applications ? »**
>
> Oui : le code produit est du vrai code, souvent avec les mêmes technologies que les professionnels. Ce qui fait la différence, c'est la qualité de la description, des vérifications, de la sécurité et de la maintenance. C'est votre rôle.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : IA ou maître d'ouvrage ?

Tâche	L'IA ou moi ?
Écrire le code du formulaire de rendez-vous	
Décider des horaires de rendez-vous disponibles	
Vérifier que le formulaire marche sur un vieux téléphone	
Proposer trois designs différents	
Garantir au client que ses données sont protégées	

🟡 Je m'entraîne

Exercice 2 : mon poste de travail

- ☐ J'ai une adresse e-mail dédiée, avec double authentification.
- ☐ J'ai un compte sur un constructeur d'applications.
- ☐ J'ai un compte GitHub.
- ☐ Mes mots de passe sont dans mon gestionnaire.
- ☐ J'ai créé mon journal de bord.

🔴 Je me lance

Exercice 3 : trois idées

Listez trois applications simples qui rendraient service dans votre entourage (famille, quartier, association, travail). Pour chacune : pour qui, quel problème elle résout, et pourquoi personne ne l'a encore faite.

🧩 CAS PRATIQUE

Agence ou vibe coding ?

Koffi hésite. Comparez les deux options pour son application de rendez-vous.

1. Quels sont les avantages de l'agence ? Ses inconvénients ?
2. Quels sont les avantages du vibe coding avec Yao ? Ses risques ?
3. Quelles questions Koffi doit-il poser à Yao avant de lui confier le projet ?
4. Dans quel cas conseilleriez-vous quand même l'agence ?

📌 CE QUE JE RETIENS

- **Vibe coding** : décrire en langage courant, l'IA écrit le code, on vérifie et on corrige.
- Vous êtes le **maître d'ouvrage**, l'IA est le **maçon** : elle construit vite, vous décidez et vous contrôlez.
- Le cycle : **décrire, construire, vérifier, corriger, sauvegarder, publier, protéger**.
- Pour commencer : **un** constructeur d'applications dans le navigateur, et s'y tenir.
- Adresse dédiée, double authentification, gestionnaire de mots de passe, **journal de bord**.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, refaites le prompt d'essai en changeant le sujet : la page d'un artisan, d'une église, d'un club de football de votre quartier. Montrez le résultat à la personne concernée et notez sa réaction.

💬 Le mot de la fin

Koffi a regardé la page de Yao sur son téléphone, longtemps. Puis il a dit : « Si tu me fais ça proprement, sécurisé, et que tu m'expliques comment ça marche, je préfère te payer toi. » Yao a souri. Il venait de trouver son premier client. Il ne savait pas encore tout ce qu'il lui restait à apprendre.

📝 Mini-quiz : ai-je bien compris ?

1. Dans le vibe coding, le code est écrit… **a)** par vous, ligne par ligne · **b)** par l'IA, à partir de vos descriptions · **c)** par le client.
2. Le terme vibe coding a été popularisé par… **a)** Andrej Karpathy en 2025 · **b)** Bill Gates en 1990 · **c)** un opérateur téléphonique.
3. Qui est responsable de la sécurité de l'application livrée au client ? **a)** l'IA · **b)** vous · **c)** personne.
4. Pour bien débuter, il vaut mieux… **a)** essayer dix outils à la fois · **b)** maîtriser un constructeur d'applications · **c)** commencer par une très grosse application.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 2

Penser comme un programmeur, sans coder

La recette de l'attiéké-poisson, écrite pour quelqu'un qui n'a jamais cuisiné

🎯 **À la fin de ce module, je saurai…**

- **découper** une idée en étapes précises ;
- reconnaître les briques de toute logique : **séquence**, **condition**, **répétition**, **données** ;
- décrire les besoins avec des **récits utilisateurs** ;
- rédiger un **cahier des charges** d'une page : utilisateurs, écrans, actions, données, règles ;
- définir une **première version** réaliste.

Durée conseillée	Matériel	Prérequis
Quatre heures	Papier, crayon, et votre journal de bord	Module 1

📖 LA SITUATION

L'application « qui fait tout »

Tantie Akissi a appris que Yao « fabrique des applications ». Elle l'appelle : « Mon fils, fais-moi une application pour ma tontine. Qu'elle fasse tout ! » Yao, enthousiaste, écrit à l'IA : « Crée une application de tontine complète. »

L'IA produit quelque chose de joli… et de faux : les membres versent de l'argent par carte bancaire, il y a des « taux d'intérêt », un « classement des meilleures épargnantes », et rien sur l'ordre de passage ni sur les retards. Rien de ce que Tantie Akissi fait vraiment.

> **Yao :** L'IA n'a rien compris !
>
> **M. Kouassi :** Elle a très bien compris ce que tu lui as dit, Yao. C'est-à-dire presque rien. Avant de parler au maçon, le maître d'ouvrage dessine le plan. Et pour dessiner le plan, il faut d'abord écouter le client.

💡 LE DÉCLIC

Un programme, c'est une recette

Écrivez la recette de l'attiéké-poisson pour quelqu'un qui n'a **jamais** cuisiné. Vous ne pouvez pas écrire « prépare le poisson ». Il faut dire : nettoyer, écailler, entailler, assaisonner, faire frire combien de temps, à quel feu. Un ordinateur est ce cuisinier débutant : il fait **exactement** ce qu'on lui dit, rien de plus. L'IA vous aide beaucoup, mais plus votre recette est précise, meilleur est le plat.

Les quatre briques de toute logique

Brique	Dans la recette	Dans une application
La séquence (dans l'ordre)	D'abord nettoyer le poisson, ensuite l'assaisonner, enfin le frire.	Le client choisit un article, puis indique la quantité, puis valide.
La condition (si… alors… sinon)	Si l'huile fume, baisser le feu.	Si la membre n'a pas versé avant dimanche, la marquer « en retard ».
La répétition (pour chaque…)	Pour chaque poisson, faire trois entailles.	Pour chaque membre, calculer le total versé.
Les données (les ingrédients)	Poisson, attiéké, oignons, tomates, piment.	Nom, téléphone, montant, date du versement.

Le récit utilisateur : « En tant que… je veux… afin de… »

Les professionnels décrivent chaque besoin par une phrase simple, toujours construite pareil. Elle oblige à penser à **qui**, **quoi** et **pourquoi**.

> **En tant que** trésorière, **je veux** voir en un coup d'œil qui est en retard **afin de** relancer les membres avant la réunion.
>
> **En tant que** membre, **je veux** voir l'historique de mes versements **afin d'**éviter les disputes.
>
> **En tant que** membre, **je veux** connaître la date à laquelle je prendrai la tontine **afin d'**organiser mes dépenses.

La première version : le pagne avant la robe

On ne coud pas une robe de mariée en une fois : on fait d'abord un patron, on essaie, on ajuste. Une application aussi se construit par étapes. La **première version** (les professionnels disent **MVP**, produit minimum viable) contient le **strict nécessaire** pour être utile. Le reste viendra ensuite, quand les vrais utilisateurs auront donné leur avis.

PAS À PAS

Étape 1 : écouter le client

Avant tout prompt, posez ces questions et notez les réponses, avec les mots du client.

Question	Réponse de Tantie Akissi
Quel problème voulez-vous régler ?	« Les disputes sur qui a payé, et le cahier illisible. »
Qui va utiliser l'application ?	« Moi la trésorière, et les trente membres. »

Question	Réponse de Tantie Akissi
Comment faites-vous aujourd'hui ?	« Un cahier, puis Excel depuis un an. Les membres paient en espèces ou par Mobile Money. »
Qu'est-ce qui serait un succès ?	« Plus aucune dispute à la réunion. »
Sur quel appareil ?	« Nos téléphones. Certaines ont des petits téléphones avec peu de données. »
Quelles règles faut-il respecter ?	« 10 000 F par semaine, versement avant dimanche, une membre prend la tontine chaque semaine selon l'ordre tiré au sort. »

Étape 2 : le cahier des charges d'une page

Rubrique	Exemple : l'application de la tontine
Objectif	Suivre les versements de la tontine et l'ordre de passage, sans dispute.
Utilisateurs	La trésorière (administratrice) ; les membres (consultation de leur propre situation).
Écrans	1. Connexion · 2. Tableau de bord de la trésorière · 3. Saisie d'un versement · 4. Liste des membres · 5. Ma situation (membre) · 6. Calendrier des passages.
Actions	La trésorière ajoute un versement, ajoute ou désactive une membre, marque la semaine comme versée à la bénéficiaire. La membre consulte, sans rien modifier.
Données	Membre (nom, téléphone, rang de passage) ; Versement (membre, date, montant, moyen) ; Semaine (numéro, bénéficiaire, date).
Règles	Cotisation 10 000 F ; « en retard » si pas de versement pour la semaine au dimanche 23 h 59 ; une membre ne voit que ses propres versements.
Contraintes	Utilisable sur petit téléphone, légère, en français simple, sans paiement en ligne dans la première version.
Hors première version	Paiement Mobile Money intégré, rappels automatiques par SMS, plusieurs tontines.

Étape 3 : dessiner les écrans sur papier

1. Pour chaque écran, dessinez un rectangle de la taille d'un téléphone.
2. Placez les éléments : titres, boutons, listes, champs. Pas besoin d'être beau : des boîtes et des flèches suffisent.
3. Reliez les écrans par des flèches : « je clique ici, j'arrive là ».
4. Montrez les dessins au client. C'est beaucoup moins cher de corriger un dessin que du code.

🖼️ **ILLUSTRATION**

Croquis à la main de trois écrans de téléphone reliés par des flèches : « Connexion » → « Tableau de bord » (total collecté, liste des retards en rouge, bouton « + Versement ») → « Saisie d'un versement » (membre, montant, date, bouton Enregistrer).

Étape 4 : vérifier avec les cas particuliers

C'est là que se cachent les vraies difficultés. Posez-vous la question « et si… ? » :

- Et si une membre paie deux semaines d'un coup ?
- Et si une membre quitte la tontine en cours d'année ?
- Et si la trésorière se trompe de montant ?
- Et si deux membres ont le même nom ?
- Et si la trésorière perd son téléphone ?

Notez la réponse du client pour chacun : ce sont des **règles** à ajouter au cahier des charges.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
🌫️ « Fais-moi une application qui fait tout »	L'IA invente des besoins qui ne sont pas les vôtres.	Un cahier des charges d'une page, avec des récits utilisateurs.
🏰 Vouloir tout dans la première version	Projet interminable, jamais livré.	Le strict nécessaire ; le reste plus tard.
🙈 Ne pas écouter les vrais utilisateurs	Une belle application que personne n'utilise.	Questions, dessins, essais avec eux.
🤷 Oublier les cas particuliers	L'application casse au premier imprévu.	La série des « et si… ? ».
🔤 Utiliser son jargon avec le client	Le client ne comprend pas et n'ose pas le dire.	Ses mots à lui, dans le cahier des charges.

🙋 Les questions qu'on n'ose pas poser

« C'est long, tout ce travail avant de commencer ! »

Une heure de cahier des charges économise souvent dix heures de corrections. Et ce document devient le cœur de votre premier prompt (module 4).

> **« Je peux demander à l'IA de m'aider à écrire le cahier des charges ? »**
>
> Oui, c'est même un très bon usage : donnez-lui vos notes d'entretien et demandez-lui de les organiser selon les rubriques, puis de lister les cas particuliers oubliés. Mais les décisions restent celles du client.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : les briques

Pour chaque phrase, indiquez la brique : séquence, condition, répétition ou données.

Phrase	Brique
Pour chaque commande du jour, imprimer un bon de livraison.	
Si le stock est inférieur à 5, afficher l'article en rouge.	
Nom du client, adresse, téléphone, articles commandés.	
Le client remplit le formulaire, puis reçoit un SMS de confirmation.	

🟡 Je m'entraîne

Exercice 2 : mes récits utilisateurs

Écrivez cinq récits utilisateurs pour l'application de rendez-vous de Koffi : deux pour les clients, trois pour Koffi.

1. En tant que .....
2. En tant que .....
3. En tant que .....
4. En tant que .....
5. En tant que .....

🔴 Je me lance

Exercice 3 : mon cahier des charges

Choisissez l'une de vos trois idées du module 1. Interrogez une vraie personne concernée, puis rédigez le cahier des charges d'une page et dessinez les écrans. Ce sera votre projet personnel pour la suite du livre.

🧩 CAS PRATIQUE

Le cahier des charges de Koffi

Koffi décrit son besoin : « Mes clients doivent pouvoir réserver un créneau pour la saisie de documents ou la mise en page de CV, du lundi au samedi, de 8 h à 18 h, par tranches de 30 minutes. Je veux voir mon planning sur mon téléphone, et que le client reçoive une confirmation. Parfois je ferme pour un deuil ou une fête. »

1. Rédigez le cahier des charges d'une page (objectif, utilisateurs, écrans, actions, données, règles, contraintes, hors première version).
2. Listez au moins cinq cas particuliers et proposez une règle pour chacun.
3. Qu'est-ce qui doit être dans la première version, et qu'est-ce qui peut attendre ?

📌 CE QUE JE RETIENS

- Un programme est une **recette** pour un cuisinier qui fait exactement ce qu'on lui dit.
- Quatre briques : **séquence**, **condition**, **répétition**, **données**.
- Les besoins s'écrivent en **récits utilisateurs** : En tant que… je veux… afin de…
- Le **cahier des charges** d'une page : objectif, utilisateurs, écrans, actions, données, règles, contraintes, hors première version.
- Dessiner les écrans **sur papier** et les montrer au client.
- Une **première version** minimale, et la série des **« et si… ? »**.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, interrogez une personne de votre entourage sur un problème de son activité, avec les six questions de l'étape 1. Rédigez son cahier des charges et faites-le-lui valider.

💬 Le mot de la fin

Yao est retourné voir Tantie Akissi avec un cahier et un crayon. Il a posé ses six questions, dessiné six écrans, et noté onze « et si… ? ». Tantie Akissi a regardé les dessins et a dit : « Là, c'est ma tontine. » Elle a ajouté en riant : « Mais pas de classement des meilleures épargnantes, hein, sinon les disputes vont recommencer ! »

📝 Mini-quiz : ai-je bien compris ?

1. « Si la membre n'a pas payé avant dimanche, la marquer en retard » est une… **a)** répétition · **b)** condition · **c)** donnée.
2. Un récit utilisateur se construit ainsi : **a)** En tant que… je veux… afin de… · **b)** Bonjour… merci… au revoir · **c)** Titre… sous-titre… conclusion.
3. La première version d'une application doit contenir… **a)** toutes les idées possibles · **b)** le strict nécessaire pour être utile · **c)** seulement le logo.

4. Il vaut mieux corriger une erreur de conception… **a)** sur le dessin papier · **b)** après la mise en ligne · **c)** jamais.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 3

Comprendre ce que l'IA fabrique

La vitrine, l'arrière-boutique, le cahier de stock et le bailleur

🎯 **À la fin de ce module, je saurai…**

- expliquer les grandes parties d'une application web : **interface**, **serveur**, **base de données**, **API** ;
- comprendre ce que sont un **hébergement** et un **nom de domaine** ;
- reconnaître **HTML**, **CSS** et **JavaScript** sans les écrire ;
- se repérer dans les **fichiers** d'un projet ;
- ouvrir les **outils de développement** du navigateur pour observer une page.

Durée conseillée	Matériel	Prérequis
Trois heures	Un ordinateur connecté, le navigateur Chrome ou Edge	Modules 1 et 2

📖 LA SITUATION

Les mots de l'IA

Yao demande à l'IA d'ajouter des comptes à l'application de la tontine. Elle répond : « Pour cela, je vais connecter un backend avec une base de données, créer une table des membres avec des politiques d'accès au niveau des lignes, et ajouter l'authentification. Je vais aussi stocker la clé d'API dans les variables d'environnement. Voulez-vous continuer ? »

Yao : Je clique sur « Oui » ? Je ne comprends pas un mot sur deux.

M. Kouassi : Un maître d'ouvrage n'a pas besoin de savoir couler le béton. Mais il doit savoir ce qu'est une fondation, une dalle et un mur porteur. Sinon, il signe n'importe quoi. On va faire le tour du chantier.

💡 LE DÉCLIC

Une application web, c'est une boutique

🖼️ ILLUSTRATION

Schéma d'une boutique en coupe : la vitrine côté rue (interface), l'arrière-boutique avec le gérant (serveur), le cahier de stock dans le tiroir (base de données), le coursier qui fait les liaisons avec la banque et le transporteur (API), l'enseigne avec l'adresse (nom de domaine), et le bailleur qui loue le local (hébergeur).

Dans la boutique…	Dans l'application	Le mot technique
La **vitrine** et le comptoir : ce que voit le client	Les pages, les boutons, les formulaires	**Interface** ou **front-end**
L'**arrière-boutique** et le gérant : il reçoit les demandes, vérifie, décide	Le programme qui traite les demandes et applique les règles	**Serveur** ou **back-end**
Le **cahier de stock** et le registre des clients	L'endroit où toutes les informations sont rangées	**Base de données**
Le **coursier** qui va à la banque ou chez le transporteur, avec un bon de commande	La porte par laquelle deux programmes s'échangent des demandes	**API**
Le **badge** à l'entrée de l'arrière-boutique	L'inscription, la connexion, les droits de chacun	**Authentification**
Le **bailleur** qui loue le local	L'entreprise dont les ordinateurs font tourner l'application	**Hébergeur**
L'**enseigne** et l'adresse	L'adresse de l'application, par exemple pagnesakissi.ci	**Nom de domaine**

La base de données : le cahier à colonnes

Une base de données ressemble à un classeur Excel bien tenu : des **tables** (comme des feuilles), avec des **colonnes** (nom, téléphone, montant) et une **ligne** par élément (une membre, un versement). Les tables sont **reliées** : chaque versement indique **à quelle membre** il appartient, comme on écrirait le numéro de la membre sur chaque reçu.

Table « membres »		
id	**nom**	**téléphone**
1	Akissi Kouamé	07…
2	Bintou Traoré	05…

Table « versements »			
id	**membre_id**	**montant**	**date**
1	2	10000	03/11/2026
2	1	10000	04/11/2026

Le versement n° 1 porte **membre_id = 2** : il appartient à Bintou. C'est ce qu'on appelle une **relation**.

Les trois langues de la vitrine

Langage	Son rôle	Comme pour une maison…	Il ressemble à…
HTML	La structure : titres, paragraphes, images, boutons	Les murs et les pièces	Pagnes Akissi
CSS	L'apparence : couleurs, tailles, disposition	La peinture et la décoration	color: orange;
JavaScript	Le comportement : ce qui se passe quand on clique	L'électricité et les interrupteurs	onClick={commander}

Beaucoup d'outils d'IA utilisent **React**, une manière moderne d'écrire des interfaces en JavaScript, et **Tailwind**, une manière rapide d'écrire le CSS. Vous verrez souvent ces noms : ce sont des outils de professionnels, et c'est une bonne nouvelle.

PAS À PAS

Étape 1 : faire le tour des fichiers d'un projet

Dans votre constructeur d'applications, ouvrez le mode **Code** (souvent un onglet ou une icône **</>**). Vous verrez une arborescence de fichiers, comme dans l'Explorateur de Windows.

Ce que vous voyez	Ce que c'est
Un dossier **src** (pour « source »)	Le code de l'application
Des fichiers **.tsx**, **.jsx**, **.js**, **.ts**	Les pages et les composants, en JavaScript ou TypeScript
Des fichiers **.css**	Les styles
Un fichier **package.json**	La liste des « ingrédients » (bibliothèques) utilisés par le projet
Un fichier **README.md**	La notice du projet, à lire en premier
Un fichier **.env** (parfois caché)	Les **réglages secrets** : clés, mots de passe. Il ne doit jamais être publié (module 10).

Étape 2 : demander à l'IA d'expliquer

Le meilleur professeur de code, c'est l'IA elle-même. Ouvrez un fichier et demandez :

> « Explique-moi ce fichier comme à quelqu'un qui ne sait pas programmer : à quoi il sert, comment il est organisé, et ce qui se passerait si je le supprimais. Utilise une comparaison avec une boutique. »

Ou, pour toute l'application : « Fais-moi un plan de l'application : quelles pages, quelles tables de données, quels services externes, et comment ils sont reliés. »

Étape 3 : observer une page avec les outils du navigateur

1. Ouvrez votre application (ou n'importe quel site) dans Chrome ou Edge.
2. Appuyez sur **F12** (ou clic droit sur la page → **Inspecter**). Un panneau s'ouvre : ce sont les **outils de développement**.
3. Onglet **Elements** (ou **Éléments**) : survolez les lignes, la partie correspondante de la page se colore. Vous voyez le HTML et le CSS de chaque élément.
4. Onglet **Console** : c'est là que s'affichent les **messages d'erreur** en rouge. Vous en aurez besoin au module 7.
5. Cliquez sur l'icône **téléphone et tablette** (mode appareil) pour voir la page comme sur un téléphone, et choisissez un modèle dans la liste.

🖼️ ILLUSTRATION

Capture d'écran : une page web avec les outils de développement ouverts à droite, l'onglet Éléments montrant le HTML, le mode appareil réglé sur un téléphone, et l'onglet Console avec un message d'erreur en rouge.

> 💡 **Le conseil de M. Kouassi**
>
> « Vous n'avez pas besoin de comprendre chaque ligne. Vous devez savoir **où** regarder, **quoi** demander, et **quand** vous méfier. C'est exactement ce que fait un bon chef de chantier. »

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
✅ Cliquer « Oui » à tout ce que propose l'IA sans comprendre	On accepte des choix coûteux ou dangereux.	Demander « explique-moi avant de le faire, avec les avantages et les risques ».
🗑️ Supprimer un fichier « qui a l'air inutile »	L'application ne démarre plus.	Demander d'abord à l'IA à quoi il sert ; sauvegarder (module 8).
🔑 Ouvrir et partager le fichier .env	Les clés secrètes sont exposées.	Ne jamais copier ce fichier dans une discussion ni le publier.
📚 Vouloir tout comprendre avant de commencer	On ne commence jamais.	Comprendre au fur et à mesure, en demandant à l'IA.

🙋 Les questions qu'on n'ose pas poser

« Toutes les applications ont-elles un serveur et une base de données ? »

Non. Une simple vitrine (module 5) peut n'être que de l'interface. Dès qu'il faut **retenir** des informations (comptes, commandes, versements), il faut une base de données et souvent un serveur.

« Supabase, Firebase : qu'est-ce que c'est ? »

Ce sont des services qui fournissent, prêts à l'emploi, une base de données, l'authentification et un stockage de fichiers. Les outils de vibe coding s'y branchent souvent automatiquement. On les appelle parfois « back-end as a service ».

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : où est-ce rangé ?

Élément	Interface, serveur, base de données, API ou hébergeur ?
Le bouton orange « Commander »	
La liste de tous les versements	
La règle « en retard si pas payé avant dimanche »	
La liaison avec le service de paiement Mobile Money	
L'entreprise qui fait tourner l'application jour et nuit	

🟡 Je m'entraîne

Exercice 2 : le tour du propriétaire

- ☐ J'ouvre le mode Code de mon projet d'essai (module 1) et je repère src, package.json et README.
- ☐ Je demande à l'IA un plan de mon application.
- ☐ J'ouvre les outils de développement (F12) et je repère un élément dans l'onglet Éléments.
- ☐ J'affiche ma page en mode téléphone.

🔴 Je me lance

Exercice 3 : les tables de mon projet

Pour votre projet personnel (module 2), dessinez les tables de données : nom de chaque table, ses colonnes, et les relations entre elles.

🧩 CAS PRATIQUE

Décoder la réponse de l'IA

Reprenez la réponse de l'IA dans la situation de départ : « connecter un backend avec une base de données, créer une table des membres avec des politiques d'accès au niveau des lignes, ajouter l'authentification, stocker la clé d'API dans les variables d'environnement ».

1. Traduisez chaque élément en langage « boutique ».
2. Lesquels sont nécessaires pour le cahier des charges de la tontine ? Pourquoi ?
3. Quelles questions Yao doit-il poser à l'IA avant d'accepter ?

📌 CE QUE JE RETIENS

- **Interface** = la vitrine ; **serveur** = l'arrière-boutique ; **base de données** = le cahier de stock ; **API** = le coursier.
- **Hébergeur** = le bailleur ; **nom de domaine** = l'enseigne et l'adresse.
- Base de données : des **tables**, des **colonnes**, des **lignes**, et des **relations**.
- **HTML** = les murs, **CSS** = la peinture, **JavaScript** = l'électricité.
- Le fichier **.env** contient les secrets : on ne le partage jamais.
- **F12** : les outils de développement, la **Console** pour les erreurs, le mode **téléphone**.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, ouvrez les outils de développement sur trois sites que vous utilisez souvent, et repérez la Console et le mode téléphone. Expliquez à quelqu'un, avec l'image de la boutique, comment fonctionne une application.

💬 Le mot de la fin

Yao a relu la réponse de l'IA. Cette fois, il a écrit : « Avant de continuer, explique-moi les politiques d'accès au niveau des lignes avec l'exemple de la tontine. » L'IA a répondu : « Cela garantit que chaque membre ne voit que ses propres versements. » Yao a souri : c'était exactement la règle de Tantie Akissi.

📝 Mini-quiz : ai-je bien compris ?

1. La partie de l'application que voit l'utilisateur s'appelle… **a)** le serveur · **b)** l'interface · **c)** la base de données.
2. Une API est comparable… **a)** au coursier qui transmet les demandes entre deux programmes · **b)** à la peinture · **c)** au bailleur.
3. Le CSS sert à… **a)** la structure · **b)** l'apparence · **c)** le paiement.
4. Le fichier .env contient… **a)** les images · **b)** les réglages secrets · **c)** la notice.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 4

L'art de parler à l'IA pour coder

Commander chez le couturier : plus on est précis, moins on retouche

🎯 **À la fin de ce module, je saurai…**

- rédiger un **premier prompt** complet à partir du cahier des charges ;
- avancer **par petites touches**, une demande à la fois ;
- utiliser le **mode plan** : faire réfléchir l'IA avant qu'elle construise ;
- donner des **références visuelles** et des **captures d'écran** ;
- se constituer une **bibliothèque de prompts** réutilisables.

Durée conseillée	Matériel	Prérequis
Quatre heures	Votre constructeur d'applications, votre cahier des charges (module 2)	Modules 1 à 3

📖 LA SITUATION

Le couturier et la robe de Mariam

Pour le mariage de sa cousine, Mariam avait dit au couturier : « Fais-moi une belle robe. » Résultat : une robe rose à volants, magnifique… pour quelqu'un d'autre. Trois retouches, deux disputes, une robe finalement correcte, la veille au soir.

Le mois suivant, elle est revenue avec une photo d'un modèle, le tissu, ses mesures, la date, et une phrase : « Comme sur la photo, mais manches trois-quarts et sans traîne. » Robe parfaite du premier coup.

> **Mariam :** Avec l'IA, c'est pareil, non ?
>
> **M. Kouassi :** Exactement pareil, Mariam. L'IA est un couturier très rapide. Si vous lui donnez un modèle, des mesures et des consignes claires, il coud juste. Sinon, il improvise… et il improvise très bien, mais pas forcément ce que vous voulez.

💡 LE DÉCLIC

Le bon de commande du couturier

Le premier prompt d'une application, c'est le **bon de commande** : il reprend le cahier des charges sous une forme que l'IA comprend bien. Ensuite, chaque demande suivante est une **retouche** précise.

Rubrique du premier prompt	Ce qu'on y met
But	Ce que fait l'application, pour qui, en une ou deux phrases.

Rubrique du premier prompt	Ce qu'on y met
Utilisateurs et rôles	Qui l'utilise, et qui a le droit de faire quoi.
Écrans	La liste des pages, et ce que chacune contient.
Données	Les informations à enregistrer, et leurs liens.
Règles	Les règles de gestion et les cas particuliers.
Style	L'ambiance, les couleurs, une référence de site ou une capture.
Contraintes	Téléphone d'abord, langue, légèreté, ce qu'il ne faut **pas** faire.
Méthode	« Présente d'abord ton plan, puis construis seulement la première étape. »

Les cinq règles des retouches

1. **Une seule demande à la fois.** « Change la couleur du bouton » puis, une fois vérifié, « ajoute un filtre ». Si vous demandez cinq choses, l'IA en ratera deux et vous ne saurez pas lesquelles.
2. **Dites où.** « Sur la page Catalogue, dans la carte de chaque produit, sous le prix… »
3. **Dites quoi, précisément.** Pas « c'est moche », mais « le texte est trop petit sur téléphone, mets-le en 16 pixels au moins ».
4. **Dites ce qu'il ne faut pas toucher.** « Ne modifie rien d'autre que ce bouton. »
5. **Vérifiez avant de continuer.** Une retouche validée, puis la suivante.

> 🔑 **Le mode plan : réfléchir avant de construire**
>
> La plupart des outils permettent de **discuter sans modifier le code** (un mode « plan », « discussion » ou « chat »). Utilisez-le pour toute demande importante : « Je veux ajouter les comptes utilisateurs. Ne code rien pour l'instant : propose-moi deux façons de faire, avec leurs avantages et leurs risques. » Vous choisissez, puis vous lancez la construction.

🖱️ PAS À PAS

Étape 1 : transformer le cahier des charges en premier prompt

1. Reprenez le cahier des charges du module 2.
2. Demandez de l'aide à un assistant d'IA : « Voici mon cahier des charges. Transforme-le en premier prompt pour un outil de création d'applications, avec les rubriques But, Utilisateurs, Écrans, Données, Règles, Style, Contraintes, Méthode. Pose-moi des questions si quelque chose est flou. »
3. Relisez et corrigez le prompt obtenu : c'est **votre** commande.
4. Terminez toujours par : « Présente d'abord ton plan et pose-moi tes questions avant de commencer. »

Étape 2 : donner des références visuelles

- Une **capture d'écran** d'un site dont vous aimez le style (**Windows + Maj + S**, puis collez-la dans la discussion) : « Inspire-toi de la mise en page de cette capture, sans copier le contenu. »
- Vos **croquis papier** du module 2, photographiés.
- Des **couleurs précises** : « orange #EA7822, vert #187A52, fond crème #F6EFE6 ».
- Une **capture de ce qui ne va pas**, avec la zone entourée : « Ici, le texte déborde sur téléphone. »

Étape 3 : les modèles de retouches

Je veux...	Modèle de demande
Ajouter une fonction	« Sur la page [X], ajoute [fonction] qui permet à [qui] de [quoi]. Quand [action], il doit se passer [résultat]. Ne modifie rien d'autre. »
Corriger l'apparence	« Sur téléphone, dans [zone], [problème précis]. Je voudrais [résultat attendu]. »
Comprendre avant d'agir	« Ne code rien. Explique-moi comment tu ferais [X], quels fichiers tu modifierais et quels sont les risques. »
Simplifier	« Cette page est trop chargée. Propose une version plus simple qui garde seulement [éléments essentiels]. »
Vérifier	« Relis le code de [fonction] et liste les cas où elle pourrait ne pas marcher. »
Revenir en arrière	« Annule la dernière modification et reviens à la version où [ce qui marchait]. »

Étape 4 : tenir sa bibliothèque de prompts

Dans votre journal de bord, gardez les prompts qui ont **bien marché**, classés par usage (premier prompt, retouche visuelle, ajout de données, correction d'erreur, sécurité). Vous les adapterez d'un projet à l'autre. L'annexe « La bibliothèque de prompts », à la fin de ce livre, vous en offre une première série.

Étape 5 : les instructions permanentes du projet

Beaucoup d'outils permettent d'enregistrer des **instructions de projet** (appelées « connaissances », « règles » ou « instructions personnalisées ») que l'IA relit à chaque demande. Mettez-y ce qui ne change pas :

> « Application en français, pour des utilisatrices ivoiriennes, souvent sur de petits téléphones avec peu de données. Montants en F CFA, sans décimales, avec espace des milliers. Dates au format JJ/MM/AAAA. Interface simple, gros boutons, textes courts. Ne jamais afficher de clé secrète dans le code de l'interface. Avant toute modification importante, présenter un plan. »

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
📦 Tout demander en un seul message	L'IA en fait la moitié, mal.	Une demande, une vérification.
😤 « Ça ne marche pas, corrige ! »	L'IA devine, et casse parfois autre chose.	Décrire le symptôme précis (module 7).
🔄 Recommencer le projet à zéro à chaque difficulté	On perd tout le travail validé.	Revenir à la dernière version qui marchait.
🗒️ Oublier de dire ce qu'il ne faut pas toucher	L'IA « améliore » aussi ce qui allait bien.	« Ne modifie rien d'autre. »
💸 Multiplier les petits messages inutiles	Beaucoup d'outils comptent les messages ou les crédits.	Réfléchir en mode plan, puis une demande claire.

🙋🏾 Les questions qu'on n'ose pas poser

« Faut-il écrire ses prompts en anglais ? »

Non : les outils comprennent très bien le français. Écrivez dans la langue où vous êtes le plus précis. Gardez simplement les termes techniques tels qu'ils sont (bouton, page, base de données, API).

« L'IA me répond qu'elle a corrigé, mais rien n'a changé. Pourquoi ? »

Elle se trompe parfois sur ce qu'elle a fait. Vérifiez toujours vous-même dans l'aperçu, rafraîchissez la page, et si besoin demandez : « Montre-moi exactement les lignes que tu as modifiées. »

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : réécrire des retouches vagues

« C'est moche. » → .....

« Ajoute des trucs pour les clients. » → .....

« Le menu bug. » → .....

🟡 Je m'entraîne

Exercice 2 : le premier prompt de la tontine

À partir du cahier des charges de la tontine (module 2), rédigez le premier prompt complet avec les huit rubriques. Ne lancez pas encore la construction : nous le ferons au module 6.

🔴 Je me lance

Exercice 3 : ma bibliothèque

Créez dans votre journal de bord votre bibliothèque de prompts avec au moins dix modèles, classés par usage, et rédigez les instructions permanentes de votre projet personnel.

🧩 CAS PRATIQUE

Les demandes de Koffi

Koffi teste la première version de son application de rendez-vous et envoie d'un coup à l'IA : « Le calendrier est trop petit, il faut aussi que je puisse bloquer des jours, et les clients doivent recevoir un message, et mets mon logo, et le bouton est bizarre sur mon téléphone. »

1. Combien de demandes différentes contient ce message ?
2. Dans quel ordre les traiter, et pourquoi ?
3. Réécrivez chacune selon les cinq règles des retouches.
4. Laquelle mérite de passer d'abord par le mode plan ?

📌 CE QUE JE RETIENS

- Le **premier prompt** est le bon de commande : But, Utilisateurs, Écrans, Données, Règles, Style, Contraintes, Méthode.
- « **Présente d'abord ton plan** » avant toute construction importante.
- Retouches : **une à la fois**, dire **où**, **quoi**, et **ce qu'il ne faut pas toucher**, puis **vérifier**.
- Des **références visuelles** : captures, croquis, couleurs précises.
- Une **bibliothèque de prompts** et des **instructions permanentes** de projet.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, pour votre projet personnel, rédigez le premier prompt complet, faites-le relire par votre binôme comme s'il était l'IA (« qu'est-ce que tu ne comprends pas ? »), puis lancez la construction de la première étape seulement.

💬 Le mot de la fin

Mariam a montré à Yao la photo de sa robe parfaite. Yao l'a collée dans son journal de bord, à la première page, avec cette légende : « Un modèle, des mesures, une retouche à la fois. »

📝 Mini-quiz : ai-je bien compris ?

1. Pour une modification importante, je demande d'abord à l'IA… **a)** de tout coder vite · **b)** de présenter un plan sans coder · **c)** de recommencer le projet.
2. Une bonne retouche contient… **a)** cinq demandes à la fois · **b)** où, quoi, et ce qu'il ne faut pas toucher · **c)** seulement « corrige ».
3. Les instructions permanentes servent à… **a)** répéter à chaque demande les règles fixes du projet · **b)** payer l'abonnement · **c)** effacer l'historique.
4. L'IA dit avoir corrigé mais rien n'a changé. Je… **a)** la crois · **b)** vérifie moi-même et lui demande les lignes modifiées · **c)** abandonne.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 5

Ma première application de A à Z

La vitrine de Pagnes Akissi, avec commande sur WhatsApp

🎯 **À la fin de ce module, je saurai…**

- construire une **vitrine** complète de plusieurs pages, du prompt à la version publiable ;
- adapter l'application au **téléphone** (responsive) ;
- créer un **lien de commande WhatsApp** prérempli ;
- soigner les **images**, les **textes** et le **référencement** de base ;
- faire **tester** l'application par de vrais utilisateurs.

Durée conseillée	Matériel	Prérequis
Six heures	Votre constructeur d'applications, des photos de produits (module 9 du Tome 2)	Modules 1 à 4

📖 LA SITUATION

Le site qui manquait

Pagnes Akissi a une vitrine WhatsApp Business et une page Facebook. Mais les clientes de Paris demandent : « Vous avez un site ? » Une agence de mariage veut un lien à mettre dans sa brochure. Et Tantie Akissi voudrait que quelqu'un qui cherche « pagne wax Adjamé » sur Google la trouve.

> **Tantie Akissi :** Yao, mon fils, fais-moi un vrai site. Pas trop compliqué : mes pagnes, mes prix, et on commande sur WhatsApp. Comme ça je réponds depuis le marché.
>
> **M. Kouassi :** C'est le projet parfait pour un premier chantier, Yao : utile, limité, et sans données sensibles. On va le faire de A à Z, proprement.

💡 LE DÉCLIC

Une vitrine, c'est une boutique sans arrière-boutique

Une **vitrine** (on dit aussi site vitrine ou « landing page » quand il n'y a qu'une page) n'a besoin ni de comptes ni de base de données : elle **montre** et elle **renvoie** vers WhatsApp pour la commande. C'est le projet idéal pour apprendre : rapide, peu risqué, immédiatement utile.

Le lien WhatsApp prérempli

WhatsApp propose des liens qui ouvrent directement une discussion avec un numéro, avec un message déjà écrit. Le format est **https://wa.me/** suivi du numéro au format international, **sans le +, sans espaces**, puis **?text=** et le message.

https://wa.me/2250700000000?text=Bonjour, je veux commander le pagne Wax hollandais bleu

Le numéro : 225 (indicatif de la Côte d'Ivoire) suivi des dix chiffres. L'IA se chargera d'encoder correctement les espaces et les accents du message.

Responsive : une boutique qui se range toute seule

Une page **responsive** s'adapte à la taille de l'écran, comme un étal qu'on réorganise selon la place : trois pagnes côte à côte sur une grande table, un seul par ligne sur une petite. En Côte d'Ivoire, la grande majorité des visiteurs viendront d'un **téléphone** : on conçoit pour le téléphone **d'abord**.

🖱️ **PAS À PAS**

Étape 1 : le prompt de départ

✍️ **Prompt**

But : un site vitrine pour Pagnes Akissi, boutique de pagnes au marché d'Adjamé (Abidjan), pour présenter les produits et recevoir les commandes sur WhatsApp.

Pages : Accueil (accroche, trois pagnes vedettes, avis de clientes, bouton WhatsApp) ; Catalogue (grille de produits filtrable par catégorie : Wax hollandais, Wax ordinaire, Pagne baoulé, Foulards) ; À propos (l'histoire de Tantie Akissi, trente ans au marché) ; Contact (adresse, horaires, carte, bouton WhatsApp).

Produits : nom, catégorie, prix en F CFA, photo, courte description. Mets douze produits d'exemple ; je remplacerai les photos.

Commande : sous chaque produit, un bouton vert « Commander sur WhatsApp » qui ouvre wa.me/2250700000000 avec le message « Bonjour Tantie Akissi, je veux commander : [nom du produit] ([prix] F) ».

Style : chaleureux, couleurs orange, vert, crème ; motif discret inspiré du tissage baoulé ; grandes photos ; police lisible.

Contraintes : conçu d'abord pour le téléphone ; rapide à charger ; tout en français ; aucun compte, aucun paiement en ligne.

Présente-moi d'abord ton plan, puis construis la page d'Accueil seulement.

Étape 2 : construire page par page

1. Validez le plan, faites construire l'**Accueil**, vérifiez-le sur ordinateur et en mode téléphone.
2. Demandez ensuite le **Catalogue**, puis **À propos**, puis **Contact** : une page par demande.
3. Après chaque page, **testez chaque bouton** : le menu, les filtres, et surtout le lien WhatsApp (sur votre téléphone, il doit ouvrir la discussion avec le bon message).
4. Sauvegardez chaque version qui marche (historique de l'outil, puis GitHub au module 8).

Étape 3 : les vraies photos et les vrais textes

- Remplacez les images d'exemple par les **vraies photos** (Tome 2, module 9). Demandez à l'IA : « Comment remplacer la photo du produit [nom] ? » : selon l'outil, on glisse l'image dans la discussion ou dans les fichiers du projet.
- Faites **réduire le poids** des images : « Optimise les images pour un chargement rapide sur téléphone. » Une photo de 5 Mo ralentit tout ; 200 Ko suffisent souvent.
- Écrivez les **textes** avec Tantie Akissi (ou faites-les rédiger par l'IA, puis relisez-les avec elle) : c'est sa voix, pas celle d'une machine.
- Vérifiez les **prix**, le **numéro** WhatsApp, l'**adresse**, les **horaires** : une seule erreur et ce sont des clientes perdues.

Étape 4 : être trouvé sur Google (référencement de base)

> « Ajoute un référencement de base : un titre et une description pour chaque page (par exemple « Pagnes Akissi – Wax et pagnes baoulés à Adjamé, Abidjan »), des textes alternatifs pour chaque image, une balise de partage pour que le lien affiche une belle image sur WhatsApp et Facebook, et un fichier sitemap. Explique-moi ce que tu as ajouté. »

Une fois le site en ligne (module 9), créez aussi une **fiche d'établissement Google** (Google Business Profile) pour apparaître sur Google Maps : c'est souvent ce qui apporte le plus de visiteurs locaux.

Étape 5 : la recette avant la livraison

Dans le bâtiment, on fait la « réception des travaux » avant de remettre les clés. Faites-la avec cette liste :

- ☐ Toutes les pages s'affichent, sur ordinateur **et** sur deux téléphones différents (dont un petit).
- ☐ Tous les liens et boutons marchent, le lien WhatsApp ouvre le bon numéro avec le bon message.
- ☐ Les prix, numéros, adresses et horaires sont exacts.
- ☐ Les photos sont nettes et légères ; la page s'affiche vite même en 3G.
- ☐ Aucun texte d'exemple oublié (« Lorem ipsum », « Votre titre ici »).
- ☐ Trois personnes qui ne connaissent pas le projet ont réussi à passer une « commande » sans aide.

ILLUSTRATION

Trois captures côte à côte de la vitrine Pagnes Akissi : l'accueil sur ordinateur, le catalogue sur téléphone (une carte par ligne), et la discussion WhatsApp ouverte avec le message prérempli.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
📞 Un numéro WhatsApp mal formaté	Le bouton n'ouvre rien, ou ouvre le mauvais numéro.	Format 225 + numéro, sans +, sans espace ; tester sur un vrai téléphone.
🐘 Des photos de plusieurs Mo	Site lent, visiteurs qui partent, forfait des clientes consommé.	Faire optimiser les images.
🖥️ Ne tester que sur ordinateur	Le site est cassé pour la majorité des visiteurs.	Téléphone d'abord, et sur plusieurs modèles.
🤖 Laisser des textes génériques écrits par l'IA	Le site sonne faux, sans personnalité.	Les vrais mots de la cliente.
🖼️ Utiliser des photos trouvées sur Internet	Droits d'auteur, et produits qui ne sont pas ceux de la boutique.	Les vraies photos des vrais produits.

🤦🏾 Les questions qu'on n'ose pas poser

« Pourquoi ne pas faire tout de suite un vrai panier avec paiement ? »

Parce que cela demande des comptes, une base de données, un paiement sécurisé et une gestion des stocks : c'est l'objet des modules 6, 10 et 11. Pour démarrer, WhatsApp suffit très bien, et Tantie Akissi garde le contact direct avec ses clientes.

« Comment Tantie Akissi pourra-t-elle changer ses prix elle-même ? »

Avec une vitrine simple, il faut repasser par l'outil (ou par Yao). Si les changements sont fréquents, on peut ajouter plus tard une petite page d'administration reliée à une base de données, ou une feuille Google Sheets que le site lit : c'est une très bonne évolution pour la version 2.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : mon lien WhatsApp

Construisez le lien wa.me de votre propre numéro avec le message « Bonjour, je viens de votre site ». Testez-le sur votre téléphone.

Mon lien : .....

🟡 Je m'entraîne

Exercice 2 : la vitrine Pagnes Akissi

Construisez la vitrine complète en suivant les cinq étapes. Remplissez la liste de la recette.

🔴 Je me lance

Exercice 3 : une vitrine pour un vrai commerçant

Proposez à un artisan, un commerçant ou une association de votre entourage de lui construire gratuitement sa vitrine, en échange de son avis et du droit de la montrer dans votre portfolio (module 13). Suivez la méthode complète, du cahier des charges à la recette.

🧩 CAS PRATIQUE

Les retours des premières testeuses

Trois clientes testent le site de Pagnes Akissi. Voici leurs remarques.

Testeuse	Remarque
Adjoua, téléphone d'entrée de gamme	« La page met longtemps à s'afficher, et j'ai vu le logo en dernier. »
Fanta, à Paris	« J'ai cliqué sur Commander, WhatsApp m'a dit que ce numéro n'existe pas. »
Marie-Laure	« Je n'ai pas compris si les prix étaient pour 6 yards ou 12 yards. »

1. Pour chaque remarque, quelle est la cause probable ?
2. Rédigez la retouche précise à envoyer à l'IA (ou l'action à faire) pour chacune.

📌 CE QUE JE RETIENS

- Une **vitrine** : montrer et renvoyer vers WhatsApp. Idéale comme premier projet.
- Lien WhatsApp : **https://wa.me/225XXXXXXXXX?text=...**, sans +, sans espace.
- **Téléphone d'abord** (responsive), et tests sur de vrais téléphones.
- Construire **page par page**, tester chaque bouton, sauvegarder chaque version qui marche.
- Vraies photos **optimisées**, vrais textes, infos **vérifiées**, référencement de base.
- La **recette** avant la livraison, avec des testeurs qui ne connaissent pas le projet.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, faites tester votre vitrine par cinq personnes, chacune sur son propre téléphone, et corrigez au moins trois problèmes qu'elles ont trouvés.

> 💬 **Le mot de la fin**
>
> Trois semaines après la mise en ligne, une cliente de Bouaké a écrit sur WhatsApp : « Bonjour Tantie Akissi, je veux commander : Pagne baoulé tissé main (45 000 F). Je vous ai trouvée sur Google. » Tantie Akissi a montré le message à toute l'allée B. Puis elle a appelé Yao : « Maintenant, on fait la tontine. »

📝 Mini-quiz : ai-je bien compris ?

1. Pour le numéro +225 07 00 00 00 00, le lien WhatsApp commence par… **a)** wa.me/+225 07… · **b)** wa.me/2250700000000 · **c)** whatsapp.ci/07…
2. Un site responsive… **a)** s'adapte à la taille de l'écran · **b)** répond aux messages · **c)** est payant.
3. Avant de livrer, je fais tester par… **a)** moi seul · **b)** des personnes qui ne connaissent pas le projet, sur leurs téléphones · **c)** personne.
4. Une photo de produit pour le web devrait peser environ… **a)** 5 Mo · **b)** quelques centaines de Ko · **c)** 1 Go.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 6

Une vraie application avec comptes et données

La tontine : chaque membre se connecte et voit ses versements

🎯 **À la fin de ce module, je saurai…**

- ajouter une **base de données** et l'**authentification** à une application ;
- définir des **rôles** (administratrice, membre) et leurs **droits** ;
- faire respecter la règle « chacun ne voit que ses données » ;
- construire les écrans de **saisie**, de **liste** et de **tableau de bord** ;
- tester avec **plusieurs comptes** et des **données réalistes**.

Durée conseillée	Matériel	Prérequis
Huit heures, en trois séances	Votre constructeur d'applications, avec sa base de données intégrée ou un compte Supabase ou Firebase	Modules 1 à 5

📖 LA SITUATION

La tontine sur le téléphone

Tantie Akissi a adoré le classeur Excel du Tome 2. Mais les membres veulent voir leur situation **elles-mêmes**, sur leur téléphone, sans appeler la trésorière. Et Tantie Akissi veut saisir un versement depuis le marché, en dix secondes, sans ordinateur.

Tantie Akissi : Mais attention, hein : Bintou ne doit pas voir ce que Adjoua a payé. Chacune son affaire.

M. Kouassi : C'est la règle la plus importante de toute l'application, Tantie. Votre fils Yao va devoir la vérifier lui-même, pas seulement faire confiance à l'IA.

💡 LE DÉCLIC

Le registre et les badges

Une application avec des comptes, c'est un bureau avec un **registre** (la base de données) et des **badges** (les comptes). À l'entrée, on vérifie le badge : c'est l'**authentification** (« qui êtes-vous ? »). Ensuite, le badge ouvre certaines portes et pas d'autres : ce sont les **autorisations** (« qu'avez-vous le droit de faire ? »).

Rôle	Badge	Il peut…	Il ne peut pas…
Trésorière	Badge doré	Voir tous les versements, en ajouter, gérer les membres	Supprimer l'historique sans trace
Membre	Badge simple	Voir **ses** versements, sa date de passage	Voir les versements des autres, modifier quoi que ce soit
Visiteur	Pas de badge	Voir la page de connexion	Voir la moindre donnée

> 🔑 **La règle doit être dans l'arrière-boutique, pas dans la vitrine**
>
> Cacher un bouton dans l'interface ne protège rien : quelqu'un d'un peu malin peut demander directement les données au serveur. La règle « chacune ne voit que ses versements » doit être appliquée **dans la base de données elle-même**. Avec Supabase, cela s'appelle la **sécurité au niveau des lignes** (Row Level Security, ou RLS). Exigez-la, et vérifiez-la.

Les tables de la tontine

Table	Colonnes principales	Relation
profils	id, nom, téléphone, rôle (trésorière ou membre), rang de passage, active	Liée au compte de connexion
versements	id, membre (profil), semaine, montant, date, moyen (espèces, Mobile Money), saisi par	Chaque versement appartient à une membre
semaines	numéro, date, bénéficiaire (profil), remise faite (oui/non)	Chaque semaine a une bénéficiaire

🖱️ PAS À PAS

Étape 1 : choisir le mode de connexion

Mode	Avantages	Limites
E-mail et mot de passe	Simple, classique, gratuit	Beaucoup de membres n'utilisent pas d'e-mail
Lien magique par e-mail	Pas de mot de passe à retenir	Demande aussi un e-mail
Numéro de téléphone avec code SMS	Adapté à nos réalités : tout le monde a un numéro	Les SMS ont un coût, à vérifier selon le service
Compte Google	Un clic	Toutes les membres n'en ont pas

Pour la tontine, discutez-en avec Tantie Akissi. Une solution réaliste pour la première version : la trésorière crée les comptes, chaque membre se connecte avec son numéro et un code.

Étape 2 : le prompt de construction

> ✍️ **Prompt (en mode plan d'abord)**
>
> Ajoute une base de données et l'authentification à l'application de la tontine. Ne code rien pour l'instant : présente-moi le plan.
>
> **Tables** : profils (nom, téléphone, rôle « tresoriere » ou « membre », rang de passage, active), versements (membre, numéro de semaine, montant, date, moyen, saisi_par), semaines (numéro, date, bénéficiaire, remise_faite).
>
> **Règles d'accès dans la base (RLS)** : une membre ne peut lire que son propre profil et ses propres versements, et ne peut rien modifier. La trésorière peut tout lire, ajouter et modifier, mais pas supprimer un versement : elle peut seulement l'annuler avec un motif.
>
> **Écrans** : connexion ; pour la trésorière : tableau de bord (total de la semaine, retards en rouge), saisie rapide d'un versement, liste des membres ; pour la membre : « Ma situation » (total versé, retards éventuels, date de mon passage).
>
> **Règles** : cotisation 10 000 F par semaine ; une membre est « en retard » si aucun versement pour la semaine en cours le dimanche à 23 h 59.
>
> Explique-moi comment je pourrai **vérifier** moi-même que les règles d'accès fonctionnent.

1. Lisez le plan. Posez vos questions. Validez.
2. Laissez l'IA créer les tables et les règles. Selon l'outil, il faudra peut-être **connecter** un projet Supabase ou Firebase (l'outil vous guide), ou activer la base de données intégrée.
3. Construisez ensuite **un écran à la fois** : connexion, puis saisie, puis tableau de bord, puis « Ma situation ».

Étape 3 : les données de test

Demandez : « Crée des données de test réalistes : une trésorière, huit membres avec des noms ivoiriens, dix semaines de versements, avec deux membres en retard et une qui a payé deux semaines d'un coup. » Des données réalistes révèlent les problèmes que des données parfaites cachent.

Étape 4 : le test des badges

C'est le test le plus important du module. Ne le sautez jamais.

- ☐ Je me connecte comme **trésorière** : je vois tout, je peux saisir un versement.
- ☐ Je me connecte comme **Bintou** (dans une fenêtre de navigation privée) : je ne vois que ses versements.
- ☐ Comme Bintou, j'essaie d'ouvrir directement l'adresse de la page de la trésorière : je suis refusée ou redirigée.
- ☐ Je demande à l'IA : « Écris un test qui vérifie qu'une membre ne peut pas lire les versements d'une autre membre, et montre-moi le résultat. »

- ☐ Déconnectée, je ne vois aucune donnée.
- ☐ Une membre désactivée ne peut plus se connecter.

🖼️ **ILLUSTRATION**

Deux téléphones côte à côte : à gauche, le tableau de bord de la trésorière (total collecté, liste des membres avec badges verts et rouges, bouton « + Versement ») ; à droite, l'écran « Ma situation » de Bintou (total versé, prochaine échéance, date de passage).

Étape 5 : simple et rapide pour la vraie vie

- Saisie d'un versement en **trois touches** : choisir la membre (recherche par nom), montant prérempli à 10 000, Enregistrer.
- Gros boutons, textes courts, chiffres lisibles : **10 000 F**, pas « 10000.00 XOF ».
- Un message clair après chaque action : « Versement de Bintou enregistré pour la semaine 12. »
- Ajoutez l'application à l'**écran d'accueil** du téléphone (menu du navigateur → **Ajouter à l'écran d'accueil**) : elle s'ouvre comme une application.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
🙈 Cacher les données dans l'interface au lieu de les protéger dans la base	N'importe qui d'un peu malin peut les lire.	Règles d'accès dans la base (RLS), et test des badges.
🗑️ Permettre de supprimer un versement	Fraude possible, disputes sans preuve.	Annulation avec motif, historique conservé.
🧪 Tester avec un seul compte	On ne voit pas les fuites entre comptes.	Au moins trois comptes de rôles différents.
👫 Mettre les vraies données des membres dans l'outil de test	Données personnelles exposées pendant le développement.	Des données fictives tant que l'application n'est pas validée.
📵 Oublier les petits téléphones et les connexions lentes	Les membres abandonnent l'application.	Tester sur un téléphone d'entrée de gamme, en 3G.

🙋 Les questions qu'on n'ose pas poser

« Où sont stockées les données ? C'est sûr ? »

Dans la base de données du service utilisé (Supabase, Firebase, ou celle intégrée à votre outil), sur des serveurs souvent situés hors d'Afrique. Ces services sont sérieux, mais la sécurité dépend surtout de **vos** réglages : règles d'accès, mots de passe, clés secrètes (module 10). Pour des données personnelles, renseignez-vous aussi sur les obligations prévues par la loi n° 2013-450.

« Et si Tantie Akissi oublie son mot de passe ? »

Prévoyez dès le départ une procédure de récupération (lien par e-mail, ou code par SMS), et **deux** comptes trésorière : Tantie Akissi et une adjointe de confiance. Une application qui dépend d'une seule personne est fragile.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : authentification ou autorisation ?

Situation	Authentification ou autorisation ?
Bintou tape son numéro et le code reçu	
Bintou ne voit pas les versements d'Adjoua	
La trésorière peut ajouter un versement	
Un visiteur est renvoyé vers la page de connexion	

🟡 Je m'entraîne

Exercice 2 : la tontine en ligne

Construisez l'application de la tontine en suivant les cinq étapes, avec des données de test. Réussissez les six vérifications du test des badges.

🔴 Je me lance

Exercice 3 : mon projet avec des comptes

Ajoutez à votre projet personnel une base de données et des comptes, avec au moins deux rôles. Rédigez vous-même la liste des vérifications du test des badges adaptée à votre projet, et passez-la.

🧩 CAS PRATIQUE

L'audit de Koffi

Koffi teste l'application de la tontine avec le compte de Bintou. Dans l'onglet **Réseau** des outils de développement (F12), il voit passer la liste complète des versements de **toutes** les membres, même si l'écran n'affiche que ceux de Bintou.

1. Quel est le problème exactement ? Pourquoi est-il grave ?
2. Où la règle d'accès doit-elle être appliquée ?
3. Rédigez la demande à envoyer à l'IA pour corriger.
4. Comment vérifier que la correction fonctionne ?

📌 CE QUE JE RETIENS

- **Authentification** = qui êtes-vous ? **Autorisation** = qu'avez-vous le droit de faire ?
- Des **rôles** clairs : chacun ses droits.
- La règle « chacun ses données » se protège **dans la base** (RLS), pas en cachant des boutons.
- On **annule avec motif**, on ne supprime pas les données importantes.
- Données de test **réalistes**, et le **test des badges** avec plusieurs comptes, en navigation privée.
- Simple et rapide pour la vraie vie : trois touches, gros boutons, petits téléphones.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, faites tester votre application avec comptes par trois personnes, chacune avec un rôle. Demandez à l'une d'elles d'essayer volontairement de voir ce qu'elle ne devrait pas voir.

💬 Le mot de la fin

Un dimanche soir, Bintou a ouvert l'application, vu « Semaine 12 : payé » en vert, et envoyé un message vocal à Tantie Akissi : « Ma sœur, c'est la première fois que la tontine me donne la paix ! » Tantie Akissi l'a fait écouter à Yao trois fois.

📝 Mini-quiz : ai-je bien compris ?

1. Vérifier l'identité d'un utilisateur, c'est… **a)** l'authentification · **b)** l'autorisation · **c)** l'hébergement.
2. La règle « chaque membre ne voit que ses versements » doit être appliquée… **a)** en cachant les boutons · **b)** dans la base de données · **c)** dans le logo.
3. Pour tester les droits, j'utilise… **a)** un seul compte · **b)** plusieurs comptes de rôles différents, en navigation privée · **c)** le compte de la trésorière seulement.
4. Un versement saisi par erreur doit être… **a)** supprimé sans trace · **b)** annulé avec un motif, l'historique étant conservé · **c)** ignoré.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 7

Quand ça ne marche pas : réparer avec l'IA

Chez le médecin, on décrit bien ses symptômes

🎯 **À la fin de ce module, je saurai…**

- trouver et lire un **message d'erreur** (aperçu, console, journaux) ;
- décrire un problème à l'IA avec la méthode **SAVE** ;
- demander un **diagnostic** avant une correction ;
- sortir de la **boucle infernale** « corrige, ça casse ailleurs » ;
- tenir un **carnet de bugs**.

Durée conseillée	Matériel	Prérequis
Quatre heures	Votre projet, le navigateur Chrome ou Edge	Modules 1 à 6 ; Tome 2, module 11

📖 LA SITUATION

La boucle infernale de Yao

La veille de la réunion de la tontine, l'écran « Ma situation » affiche une page blanche. Yao écrit à l'IA : « Ça marche pas, corrige. » L'IA modifie trois fichiers. Maintenant, la page s'affiche, mais la connexion ne marche plus. « Corrige la connexion. » La connexion revient, mais les montants affichent « NaN F ». « Corrige les montants. » La page redevient blanche.

Deux heures, vingt-trois messages, et l'application est en pire état qu'au départ.

Yao : L'IA tourne en rond, et moi avec !

M. Kouassi : Tu ne l'aides pas, Yao. Tu lui dis « j'ai mal » sans dire où, depuis quand, ni ce que tu as mangé. Même le meilleur médecin donnerait des médicaments au hasard. Et quand un traitement empire les choses, on arrête le traitement : on revient à la dernière fois où le patient allait bien.

💡 LE DÉCLIC

Le bon patient fait le bon diagnostic

L'IA corrige très bien… ce qu'elle comprend. Votre rôle est d'être le **meilleur patient du monde** : précis, factuel, avec les résultats d'analyse. Retenez la méthode **SAVE** :

Lettre	Ce que je donne	Exemple
S	**Symptôme** : ce que je vois, exactement	« L'écran Ma situation est entièrement blanc. »

Lettre	Ce que je donne	Exemple
A	**Action** : ce que je faisais, pour le reproduire	« Je me connecte avec le compte de Bintou, puis je touche Ma situation. »
V	**Voulu** : ce qui devrait se passer	« Je devrais voir son total versé et sa date de passage. »
E	**Erreur** : le message exact (console, aperçu) et ce qui a changé récemment	« Console : TypeError: Cannot read properties of undefined (reading 'montant'). Le problème est apparu après l'ajout des semaines. »

Où trouver les messages d'erreur

Endroit	Ce qu'on y trouve
L'aperçu de l'outil	Souvent un bandeau rouge avec l'erreur, et un bouton « Corriger » ou « Try to fix »
La console du navigateur (F12 → Console)	Les erreurs de l'interface, en rouge
L'onglet Réseau (F12 → Network ou Réseau)	Les demandes au serveur qui échouent (codes 400, 401, 403, 500…)
Les journaux (logs) du service de données ou de l'hébergeur	Les erreurs du serveur et de la base de données

Code	Ce qu'il veut dire, en mots simples
401	Vous n'êtes pas connecté (pas de badge).
403	Vous êtes connecté, mais pas autorisé (badge refusé à cette porte). Souvent une règle d'accès.
404	Introuvable : la page ou la donnée n'existe pas à cette adresse.
500	Erreur dans l'arrière-boutique : le serveur a planté en traitant la demande.

PAS À PAS

Étape 1 : observer et reproduire

1. Notez précisément le symptôme et faites une **capture d'écran**.
2. Essayez de **reproduire** le problème : arrive-t-il à chaque fois ? Avec tous les comptes ? Sur téléphone et ordinateur ?
3. Ouvrez la **console** (F12) et copiez le message rouge **en entier**.
4. Demandez-vous : **qu'est-ce qui a changé** juste avant que le problème apparaisse ?

Étape 2 : demander un diagnostic, pas une correction

> ✍️ **Prompt de diagnostic (en mode plan)**
>
> « Ne modifie rien pour l'instant. **Symptôme** : [ce que je vois]. **Action** : [comment le reproduire]. **Voulu** : [ce qui devrait se passer]. **Erreur** : [message exact de la console]. **Changement récent** : [ce qui a été fait juste avant]. Explique-moi la cause la plus probable, les autres causes possibles, et la correction la plus petite possible. »

Lisez l'explication. Si elle a du sens, demandez : « Applique uniquement cette correction, sans rien modifier d'autre. » Puis **vérifiez**.

Étape 3 : sortir de la boucle infernale

Si, après **trois** tentatives, le problème persiste ou se déplace, arrêtez-vous. Continuer, c'est empiler les médicaments.

1. **Revenez** à la dernière version qui marchait (historique de l'outil, ou GitHub au module 8).
2. Demandez à l'IA d'**expliquer** le fonctionnement de la partie concernée, avant tout changement.
3. Découpez : refaites la modification en **plus petites étapes**, en vérifiant après chacune.
4. Si besoin, faites **ajouter des messages de suivi** : « Ajoute des messages dans la console qui affichent les données reçues à chaque étape de cette page. » Vous verrez où ça se casse.
5. Changez d'angle : décrivez le problème à **un autre assistant** d'IA, ou ouvrez une **nouvelle conversation** en résumant la situation : une discussion trop longue embrouille parfois l'IA.

Étape 4 : le carnet de bugs

Dans votre journal de bord, tenez un tableau simple. Il vous fera gagner des heures sur les projets suivants.

Date	Symptôme	Cause trouvée	Solution	Prompt qui a marché
12/11	Ma situation : page blanche	Semaine sans versement : montant indéfini	Afficher 0 F si aucun versement	« Gère le cas où… »

🖼️ **ILLUSTRATION**

Capture d'écran : la console du navigateur avec une erreur rouge « TypeError: Cannot read properties of undefined », la ligne concernée surlignée, et à côté la fenêtre de l'outil de vibe coding avec le prompt de diagnostic SAVE.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
😤 « Ça marche pas, corrige »	L'IA devine et modifie au hasard.	La méthode SAVE.
🔁 Enchaîner vingt corrections	Le code devient un plat de spaghettis.	Trois essais maximum, puis retour à la dernière version qui marchait.
🙈 Ne pas ouvrir la console	On se prive du message qui donne la réponse.	F12 → Console, à chaque problème.
✂️ Copier une partie du message d'erreur	L'information utile est souvent dans la partie coupée.	Copier le message en entier.
🔐 Coller des clés secrètes dans la discussion pour « aider l'IA »	Les secrets sortent de votre contrôle.	Remplacer toute valeur secrète par « [CLÉ] ».

🙋🏾 Les questions qu'on n'ose pas poser

« Le bouton « Corriger automatiquement » de mon outil, je l'utilise ? »

Oui, pour une erreur simple et isolée : il transmet le message exact à l'IA. Mais si le problème revient ou se déplace, reprenez la main avec la méthode SAVE et le mode plan.

« Comment savoir si l'erreur vient de mon code ou d'un service extérieur ? »

Regardez l'onglet Réseau : si la demande à un service (paiement, base de données) renvoie une erreur, le problème est dans la liaison ou dans le réglage de ce service. Consultez aussi la page d'état de ce service (souvent appelée « status ») : il est peut-être en panne pour tout le monde.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : SAVE ou pas SAVE ?

Réécrivez ces demandes avec la méthode SAVE (inventez les détails réalistes).

« Le formulaire bug. » → .....

« Les prix sont faux. » → .....

🟡 Je m'entraîne

Exercice 2 : le bug provoqué

Demandez à votre binôme de casser volontairement quelque chose dans une copie de votre projet (sans vous dire quoi). Trouvez le problème avec la console et la méthode SAVE, et notez-le dans votre carnet de bugs.

🔴 Je me lance

Exercice 3 : sortir de la boucle

Pour le prochain problème qui résiste à trois essais dans votre projet, appliquez les cinq étapes de l'étape 3 et racontez dans votre journal ce qui a finalement débloqué la situation.

🧩 CAS PRATIQUE

Les « NaN F » de la tontine

Sur le tableau de bord de la trésorière, le total de la semaine affiche **NaN F** depuis que Yao a ajouté la possibilité de saisir « 10 000 » avec un espace. La console n'affiche aucune erreur rouge.

1. Que signifie probablement « NaN » ? (Indice : demandez-le à l'IA.)
2. Quel changement récent est suspect ?
3. Rédigez le prompt de diagnostic SAVE.
4. Quelle correction durable proposer pour que cela ne se reproduise pas, même si quelqu'un tape « 10.000 » ou « 10 000 F » ?

📌 CE QUE JE RETIENS

- Le message d'erreur est votre meilleur ami : **aperçu**, **console** (F12), **onglet Réseau**, **journaux**.
- Méthode **SAVE** : Symptôme, Action, Voulu, Erreur (et ce qui a changé).
- D'abord un **diagnostic** (mode plan), ensuite la **plus petite** correction, puis vérifier.
- Trois essais sans succès : **revenir** à la dernière version qui marchait, découper, ajouter des messages de suivi.
- Un **carnet de bugs** : symptôme, cause, solution, prompt.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, notez dans votre carnet chaque problème rencontré, même petit. À la fin de la semaine, relisez-le : quels types d'erreurs reviennent le plus ?

> 💬 **Le mot de la fin**
>
> Yao est revenu à la version de la veille. Il a ouvert la console, lu le message, et écrit un prompt SAVE de six lignes. L'IA a répondu : « La semaine 12 n'a encore aucun versement, le montant est indéfini. » Une correction de deux lignes. Une minute. Yao a collé la phrase de M. Kouassi au-dessus de son écran : « Le bon patient fait le bon diagnostic. »

📝 Mini-quiz : ai-je bien compris ?

1. Les erreurs de l'interface s'affichent dans… **a)** la console du navigateur · **b)** la Corbeille · **c)** le README.
2. Dans SAVE, le V signifie… **a)** Version · **b)** Voulu · **c)** Vite.
3. Un code 403 signifie… **a)** page introuvable · **b)** connecté mais pas autorisé · **c)** tout va bien.
4. Après trois corrections qui ne marchent pas, je… **a)** continue jusqu'à cinquante · **b)** reviens à la dernière version qui marchait et je découpe · **c)** supprime le projet.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 8

Sauvegarder chaque étape avec GitHub

L'album photo de la construction, étage par étage

🎯 **À la fin de ce module, je saurai…**

- comprendre le principe du **contrôle de versions** ;
- relier son projet à un **dépôt GitHub** ;
- faire des **commits** clairs et lire l'**historique** ;
- revenir à une **version précédente** ;
- utiliser les **branches** pour essayer sans risque, et écrire un **README**.

Durée conseillée	Matériel	Prérequis
Trois heures	Votre compte GitHub, votre projet	Modules 1 à 7

📖 LA SITUATION

Le projet disparu

Un matin, Yao ouvre son outil de vibe coding : son compte a été suspendu « pour vérification de paiement ». Sa carte a été refusée pour l'abonnement du mois. L'application de la tontine, trois semaines de travail, est inaccessible. Les membres l'utilisent tous les jours.

Deux jours d'angoisse plus tard, le compte est rétabli. Yao a eu très peur.

Yao : Si le compte avait été fermé, j'aurais tout perdu ?

M. Kouassi : Oui, Yao. Ton travail ne doit jamais dépendre d'un seul endroit. Le code doit avoir sa maison à lui, avec l'histoire de chaque étape. Cette maison, c'est GitHub.

💡 LE DÉCLIC

L'album photo de la construction

Quand on construit une maison, certains prennent une photo à chaque étape : les fondations, le premier mur, la dalle, la toiture. Si un jour on veut savoir où passe un tuyau, on regarde l'album. **Git** est cet album pour le code : chaque étape enregistrée s'appelle un **commit**, avec une date, un auteur et un message. **GitHub** est le site où l'album est rangé, en sécurité, en ligne.

Mot	Dans l'album photo…	Dans GitHub
Dépôt (repository)	L'album d'un chantier	Le dossier en ligne qui contient tout le code et son histoire

Mot	Dans l'album photo…	Dans GitHub
Commit	Une photo datée avec une légende	Un enregistrement des modifications, avec un message
Historique	Feuilleter l'album	La liste de tous les commits
Branche	Un album de brouillon pour essayer une extension, sans toucher à l'album principal	Une version parallèle du code pour essayer
Fusion (merge)	Coller les photos du brouillon dans l'album principal	Intégrer une branche validée
README	La première page de l'album, qui explique le chantier	La notice du projet

🔑 **Trois raisons d'utiliser GitHub**

- **La sécurité** : votre code existe ailleurs que dans l'outil de vibe coding.
- **La liberté** : vous pouvez changer d'outil, d'hébergeur, ou confier le projet à quelqu'un d'autre.
- **La mémoire** : vous pouvez revenir à n'importe quelle étape, et comprendre ce qui a changé.

🖱️ PAS À PAS

Étape 1 : relier le projet à GitHub

1. Connectez-vous à **github.com** (compte créé au module 1, avec double authentification).
2. Dans votre constructeur d'applications, cherchez **GitHub** dans les réglages du projet ou l'intégration (souvent un bouton « Connect to GitHub » ou « Export to GitHub »).
3. Autorisez l'outil à accéder à votre compte GitHub, puis créez un nouveau **dépôt**. Choisissez **Private** (privé) pour un projet client.
4. Selon l'outil, chaque modification sera ensuite enregistrée automatiquement dans GitHub, ou vous devrez cliquer sur un bouton de synchronisation.
5. Ouvrez votre dépôt sur github.com : vous voyez vos fichiers.

Étape 2 : lire l'historique

1. Sur la page du dépôt, cliquez sur **Commits** (avec une petite horloge).
2. Chaque ligne est une étape : message, auteur, date. Cliquez sur une ligne pour voir **ce qui a changé** : en vert les lignes ajoutées, en rouge les lignes supprimées.
3. Si les messages sont vagues (« update », « changes »), demandez à votre outil ou à l'IA d'écrire des messages clairs : « Ajout du calendrier des passages », « Correction du décalage horaire des rendez-vous ».

🖼️ **ILLUSTRATION**

Capture d'écran de GitHub : la liste des commits d'un dépôt « tontine-allee-b » avec des messages clairs, et à côté le détail d'un commit montrant des lignes ajoutées en vert et supprimées en rouge.

Étape 3 : revenir en arrière

- **Le plus simple** : utiliser l'historique de votre outil de vibe coding pour restaurer une version, qui sera ensuite enregistrée dans GitHub.
- **Depuis GitHub** : demandez à un agent de code ou à l'IA de votre outil : « Reviens à l'état du code du commit [message ou numéro], sans perdre l'historique. » En langage Git, on parle de **revert**.
- Ne supprimez **jamais** l'historique pour « faire propre » : c'est votre assurance.

Étape 4 : les branches pour essayer sans risque

Vous voulez essayer une grosse nouveauté (le paiement Mobile Money, par exemple) sans risquer l'application que les membres utilisent ? Créez une **branche**.

1. Dans votre outil (s'il gère les branches) ou sur GitHub : créez une branche **essai-paiement**.
2. Travaillez sur cette branche ; l'application principale (branche **main**) ne bouge pas.
3. Quand tout est validé, **fusionnez** la branche dans main (sur GitHub, on passe par une **Pull Request** : une demande de fusion, qui montre toutes les modifications avant de les accepter).
4. Si l'essai échoue, abandonnez simplement la branche.

Étape 5 : le README, la notice du projet

> ✍️ **Prompt**
>
> « Rédige un fichier README.md en français pour ce projet : à quoi sert l'application, pour qui, les fonctionnalités, les technologies et services utilisés, comment la lancer, où sont configurées les variables d'environnement (sans aucune valeur secrète), et qui contacter. »

Un bon README permet à quelqu'un d'autre (ou à vous dans six mois) de reprendre le projet sans tout redécouvrir.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
🗄️ Garder son code seulement dans l'outil de vibe coding	Compte suspendu, outil fermé : tout est perdu.	Relier chaque projet à GitHub dès le début.
🌍 Mettre un projet client en dépôt public	Le code et parfois des secrets sont visibles par tous.	Dépôt privé par défaut.

Le piège	Pourquoi c'est dangereux	Le bon réflexe
🔑 Envoyer le fichier .env sur GitHub	Les clés secrètes sont exposées, des robots les cherchent en permanence.	Vérifier que .env est ignoré ; changer immédiatement toute clé publiée par erreur.
📝 Des messages de commit vagues	Impossible de retrouver la bonne étape.	Un message qui dit ce qui a changé et pourquoi.
🎰 Essayer les grosses nouveautés directement sur main	L'application des utilisateurs casse.	Une branche pour chaque essai important.

🙋🏾 Les questions qu'on n'ose pas poser

> **« GitHub, c'est pour les programmeurs. Je vais m'y perdre ? »**
>
> Vous n'utiliserez au début que trois choses : le dépôt, l'historique des commits et le README. Votre outil de vibe coding fait le reste. Et l'IA peut vous expliquer tout ce que vous voyez.

> **« À qui appartient le code que j'ai fait générer pour un client ? »**
>
> C'est une question de **contrat** (module 13). En général, on prévoit que le code devient la propriété du client à la fin du paiement, et on lui transfère le dépôt GitHub ou on l'y ajoute comme propriétaire.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : le vocabulaire

Je veux...	Le mot Git ou GitHub
L'album complet d'un projet	
Une photo datée d'une étape	
Un brouillon pour essayer	
Intégrer le brouillon validé	
La notice du projet	

🟡 Je m'entraîne

Exercice 2 : mon dépôt

- ☐ Je relie mon projet à un dépôt GitHub privé.
- ☐ Je vérifie que le fichier .env n'apparaît pas dans le dépôt.
- ☐ Je fais trois modifications et je retrouve les trois commits dans l'historique.
- ☐ Je reviens à l'avant-dernière version, puis à la dernière.
- ☐ Je fais générer un README, je le relis et je le corrige.

🔴 Je me lance

Exercice 3 : ma première branche

Créez une branche pour essayer une nouveauté dans votre projet personnel (un nouveau thème de couleurs, par exemple). Faites-la valider par votre binôme, puis fusionnez-la avec une Pull Request.

🧩 CAS PRATIQUE

Le départ de Yao

Yao part six mois en stage à Dakar. Koffi doit confier la maintenance de son application de rendez-vous à une autre personne, Awa.

1. Que doit contenir le dépôt GitHub pour qu'Awa puisse reprendre le projet ?
2. Comment donner accès à Awa sans lui donner le compte de Yao ?
3. Quelles informations ne doivent **pas** être dans le dépôt, et comment les transmettre ?

📌 CE QUE JE RETIENS

- **Git** = l'album photo du code ; **GitHub** = l'endroit où il est rangé en ligne.
- **Dépôt**, **commit**, **historique**, **branche**, **fusion** (Pull Request), **README**.
- Chaque projet relié à GitHub **dès le début**, en dépôt **privé** pour un client.
- Des messages de commit **clairs** ; on ne supprime jamais l'historique.
- Une **branche** pour chaque essai important.
- Le fichier **.env** ne va **jamais** sur GitHub.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, reliez tous vos projets à GitHub, avec un README pour chacun. Vérifiez qu'aucun secret n'a été publié.

> 💬 **Le mot de la fin**
>
> Avant de partir à Dakar, Yao a ajouté Awa comme collaboratrice du dépôt et lui a laissé un README de deux pages. La première semaine, Awa a corrigé un bug en suivant le carnet de bugs. Elle a écrit à Yao : « Ton album photo est tellement clair que j'ai l'impression d'avoir construit la maison avec toi. »

📝 Mini-quiz : ai-je bien compris ?

1. Un commit, c'est… **a)** une étape enregistrée avec un message · **b)** un mot de passe · **c)** un hébergeur.
2. Pour un projet client, le dépôt doit être… **a)** public · **b)** privé · **c)** supprimé.
3. Pour essayer une grosse nouveauté sans risque, j'utilise… **a)** une branche · **b)** le fichier .env · **c)** la Corbeille.
4. Le fichier .env sur GitHub, c'est… **a)** obligatoire · **b)** une fuite de secrets à éviter absolument · **c)** sans importance.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 9

Mettre son application en ligne

Ouvrir la boutique au public : l'adresse, l'enseigne, les clés

🎯 **À la fin de ce module, je saurai…**

- comprendre ce qu'est un **déploiement** et un **hébergeur** ;
- publier depuis son outil, ou depuis **GitHub** vers **Vercel** ou **Netlify** ;
- configurer les **variables d'environnement** de production ;
- acheter un **nom de domaine** (.ci ou .com) et le relier à l'application ;
- publier des **mises à jour** sans casser l'application.

Durée conseillée	Matériel	Prérequis
Quatre heures	Votre projet relié à GitHub, un moyen de paiement pour le nom de domaine	Modules 1 à 8

📖 LA SITUATION

L'adresse impossible à retenir

L'application de rendez-vous de Koffi fonctionne. Yao lui envoie le lien : **https://reserv-app-x7k2q9-preview.exemple.app**. Koffi le colle sur son affiche : personne n'arrive à le recopier. Pire : un soir, Yao fait un essai dans l'outil, et la version des clients casse en direct pendant une heure.

Koffi : Yao, mes clients veulent une vraie adresse, comme les grandes entreprises. Et une application qui ne s'arrête pas quand tu fais tes essais !

M. Kouassi : Deux leçons, Koffi : une enseigne lisible, c'est le nom de domaine. Et on ne repeint pas la vitrine pendant que les clients sont dans la boutique : on sépare l'atelier d'essai de la boutique ouverte au public.

💡 LE DÉCLIC

Ouvrir la boutique

Mot	Comme pour une boutique…	Ce que c'est
Déployer	Ouvrir la boutique au public	Mettre l'application en ligne, accessible à tous

Mot	Comme pour une boutique…	Ce que c'est
Hébergeur	Le bailleur du local	L'entreprise dont les serveurs font tourner l'application (Vercel, Netlify, l'hébergement intégré de votre outil…)
Préproduction (ou aperçu)	L'atelier à l'arrière, où l'on essaie	Une version de test, invisible pour les clients
Production	La boutique ouverte	La version utilisée par les vrais utilisateurs
Nom de domaine	L'enseigne et l'adresse	Une adresse lisible comme **koffiservices.ci**
DNS	L'annuaire qui indique où se trouve la boutique	Les réglages qui relient le nom de domaine à l'hébergeur
Variables d'environnement	Les clés du coffre, confiées au gérant et jamais affichées en vitrine	Les réglages secrets (clés d'API, adresses de base de données) donnés à l'hébergeur

Le .ci ou le .com ?

Extension	Pour qui	À savoir
.ci	Une activité tournée vers la Côte d'Ivoire : fierté locale, confiance des clients ivoiriens	Extension nationale, gérée par l'autorité ivoirienne (l'ARTCI, via le registre NIC.CI). Achat auprès d'un **bureau d'enregistrement accrédité**. Les conditions et les prix évoluent : vérifiez-les sur le site officiel du registre.
.com	Une activité tournée vers l'international, ou en complément	Achat facile auprès de nombreux bureaux d'enregistrement internationaux, souvent payable par carte.

> 🔑 **Le renouvellement**
>
> Un nom de domaine se **loue**, en général à l'année. S'il n'est pas renouvelé, l'application devient inaccessible, et quelqu'un d'autre peut racheter le nom. Activez le **renouvellement automatique** et notez la date dans l'agenda du client.

🖱️ PAS À PAS

Chemin A : publier depuis son outil de vibe coding

1. Cliquez sur **Publish**, **Deploy** ou **Publier** dans votre outil.

2. L'outil donne une adresse publique. Ouvrez-la sur votre téléphone, **en navigation privée**, pour voir l'application comme un nouveau visiteur.
3. Cherchez dans les réglages du projet la section **Domaine** (Domain, Custom domain) pour y relier votre nom de domaine plus tard.
4. Vérifiez comment l'outil sépare l'**aperçu** (vos essais) de la **version publiée** (vos clients) : en général, les clients ne voient vos changements qu'après un nouveau **Publish**.

Chemin B : de GitHub vers Vercel ou Netlify

Ce chemin donne plus de liberté (vous ne dépendez plus d'un seul outil) et met à jour l'application à chaque modification enregistrée dans GitHub.

1. Créez un compte sur **vercel.com** ou **netlify.com** en vous connectant avec **GitHub**.
2. **Add New Project** (ou **Add new site**) → **Import** → choisissez votre dépôt.
3. L'hébergeur détecte en général tout seul le type de projet. Avant de lancer, ouvrez **Environment Variables** et ajoutez les variables de votre fichier .env (les **noms** et les **valeurs** de production), une par une.
4. Cliquez sur **Deploy**. Après une ou deux minutes, une adresse s'affiche.
5. Désormais, chaque commit sur la branche **main** déclenche un nouveau déploiement ; chaque branche d'essai reçoit sa propre adresse d'aperçu, sans toucher à la production.

> 🖼 ILLUSTRATION
> *Capture d'écran d'un tableau de bord d'hébergeur : le projet « koffi-rendez-vous », le dernier déploiement réussi en vert, l'adresse de production, l'onglet Environment Variables et l'onglet Domains.*

Étape commune : relier le nom de domaine

1. Achetez le nom de domaine auprès d'un bureau d'enregistrement (pour le .ci, un bureau accrédité).
2. Dans l'hébergeur ou l'outil : **Domains** → **Add** → tapez **koffiservices.ci**. L'hébergeur affiche les **enregistrements DNS** à créer (souvent un enregistrement **A** et un enregistrement **CNAME** pour **www**).
3. Dans l'espace client du bureau d'enregistrement, section **DNS** ou **Zone DNS** : créez exactement ces enregistrements (copier-coller).
4. Patientez : la propagation peut prendre de quelques minutes à quarante-huit heures. Le **cadenas HTTPS** est en général ajouté automatiquement par l'hébergeur.
5. Testez **koffiservices.ci** et **www.koffiservices.ci** sur plusieurs appareils.

Publier une mise à jour sans casser

- ☐ La modification est faite sur une **branche** ou dans l'aperçu, pas directement en production.
- ☐ Je l'ai testée sur l'adresse d'aperçu, sur téléphone et ordinateur, avec plusieurs comptes.
- ☐ Les variables d'environnement nécessaires existent en production.
- ☐ Je publie à une heure creuse (pas le samedi matin pour un secrétariat).

☐ Je vérifie la production juste après, et je sais comment revenir à la version précédente (bouton **Rollback** ou **Redeploy** d'un déploiement ancien chez l'hébergeur).

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
🧪 Faire ses essais directement sur la version des clients	L'application casse en plein usage.	Aperçu ou branche d'abord, publication ensuite.
🔑 Oublier les variables d'environnement en production	L'application en ligne ne se connecte pas à la base de données.	Les ajouter dans l'hébergeur avant le déploiement.
📛 Acheter le nom de domaine au nom du prestataire	Le client ne possède pas son adresse, conflits futurs.	Le domaine au nom du client, avec ses accès.
⏰ Laisser expirer le nom de domaine	Site inaccessible, nom racheté par un autre.	Renouvellement automatique et rappel.
🧮 Ignorer les limites des offres gratuites	Application suspendue au pic d'activité, ou facture surprise.	Lire les limites ; prévoir l'offre adaptée et l'inclure dans le prix.

🙋🏾 Les questions qu'on n'ose pas poser

« Combien coûte l'hébergement ? »

Pour une petite application, les offres gratuites de Vercel, Netlify ou de votre outil suffisent souvent au début. Pour un usage commercial ou plus important, prévoyez un abonnement. Ajoutez le nom de domaine (quelques milliers à quelques dizaines de milliers de francs par an selon l'extension et le bureau). Vérifiez toujours les prix et les conditions d'utilisation commerciale.

« Mon application peut-elle être installée sur le téléphone comme une vraie application ? »

Une application web peut être ajoutée à l'écran d'accueil (module 6) et, avec quelques réglages (on parle de « PWA », application web progressive), fonctionner presque comme une application installée. La publier sur le Play Store ou l'App Store est possible, mais demande des étapes, des comptes développeur payants et des validations supplémentaires.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : remettre dans l'ordre

Remettez dans l'ordre : « Ajouter le domaine dans l'hébergeur » · « Acheter le nom de domaine » · « Tester avec et sans www » · « Créer les enregistrements DNS » · « Attendre la propagation ».

1. .....

2. .....

3. .....

4. .....

5. .....

🟡 Je m'entraîne

Exercice 2 : ma première mise en ligne

Publiez votre vitrine (module 5) par le chemin A, puis par le chemin B depuis GitHub vers Vercel ou Netlify. Comparez les deux adresses et le fonctionnement des mises à jour.

🔴 Je me lance

Exercice 3 : une mise à jour en douceur

Faites une modification visible (une nouvelle section) sur une branche, testez-la sur l'adresse d'aperçu, publiez-la, puis exercez-vous à revenir à la version précédente avec le Rollback de l'hébergeur.

🧩 CAS PRATIQUE

La mise en ligne de Koffi Services

Koffi veut koffiservices.ci, une application qui ne casse plus pendant les essais, et il veut rester propriétaire de tout.

1. Quel chemin de déploiement conseillez-vous, et pourquoi ?
2. Qui doit acheter le nom de domaine, à quel nom, et avec quel réglage ?
3. Comment organiser les essais de Yao pour ne plus casser l'application des clients ?
4. Quelle liste remettre à Koffi pour qu'il sache ce qu'il possède et ce qu'il paie chaque année ?

📌 CE QUE JE RETIENS

- **Déployer** = ouvrir la boutique. **Aperçu** = l'atelier ; **production** = la boutique ouverte.
- Chemin A : **Publish** depuis l'outil. Chemin B : **GitHub → Vercel ou Netlify**, mise à jour à chaque commit.
- Les **variables d'environnement** de production se règlent chez l'hébergeur, jamais dans le code.
- Nom de domaine : **.ci** ou **.com**, **au nom du client**, **renouvellement automatique**.
- Relier le domaine : enregistrements **DNS** (A, CNAME), patience, **HTTPS** automatique.
- Mettre à jour : tester en aperçu, publier à une heure creuse, savoir faire un **Rollback**.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, mettez en ligne votre projet personnel à une adresse publique, et faites-le utiliser par trois personnes depuis chez elles.

💬 Le mot de la fin

Sur la nouvelle affiche du secrétariat, il n'y a plus qu'une ligne sous le logo : koffiservices.ci. Le premier mois, cent quatorze rendez-vous ont été pris en ligne, dont trente-deux un dimanche soir. Et Yao fait maintenant ses essais sur une branche, à l'abri des clients.

📝 Mini-quiz : ai-je bien compris ?

1. Déployer une application, c'est… **a)** la supprimer · **b)** la mettre en ligne, accessible au public · **c)** la sauvegarder sur une clé USB.
2. Les clés secrètes de production se configurent… **a)** dans le code visible · **b)** dans les variables d'environnement de l'hébergeur · **c)** sur l'affiche.
3. Le nom de domaine d'un client doit être enregistré… **a)** au nom du prestataire · **b)** au nom du client · **c)** au nom de l'hébergeur.
4. Pour relier un nom de domaine à l'hébergeur, on modifie… **a)** les DNS · **b)** le README · **c)** la police.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 10

Une application sûre et de qualité

La boutique bien fermée le soir, et propre pour les clients

🎯 **À la fin de ce module, je saurai…**

- protéger les **clés secrètes** et comprendre pourquoi elles fuient ;
- appliquer les règles de base de la sécurité d'une application : **accès**, **saisies**, **mises à jour** ;
- respecter les **données personnelles** des utilisateurs et le cadre légal ivoirien ;
- vérifier la **qualité** : téléphone, vitesse, accessibilité, textes ;
- faire réaliser une **revue de sécurité** et passer la **liste de contrôle** avant livraison.

Durée conseillée	Matériel	Prérequis
Cinq heures	Votre projet en ligne	Modules 1 à 9 ; Tome 2, module 8

📖 LA SITUATION

La facture de 400 000 F

Un dimanche matin, Yao reçoit un e-mail d'un service d'intelligence artificielle : « Votre consommation du mois : 680 dollars. » Il n'a presque rien utilisé. En cherchant, il découvre que la **clé secrète** de ce service était écrite directement dans le code de l'interface de son application de démonstration. N'importe quel visiteur pouvait la lire avec F12. Des robots l'ont trouvée et l'ont utilisée toute la nuit.

Yao : Mais c'est l'IA qui l'a mise là ! Je lui avais juste donné la clé pour que ça marche.

M. Kouassi : L'IA veut que ça marche, Yao. La sécurité, c'est ton travail. C'est toi qui as donné la clé, et c'est toi que le client tiendra pour responsable. On va apprendre à fermer la boutique le soir.

💡 LE DÉCLIC

Les clés du coffre ne vont pas dans la vitrine

Tout ce qui est dans l'**interface** (la vitrine) est **visible** par tout visiteur, même si ce n'est pas affiché à l'écran : il suffit d'ouvrir les outils de développement. Une clé secrète (paiement, IA, envoi de SMS) doit rester dans l'**arrière-boutique** : sur le serveur, dans les **variables d'environnement** (module 9), et être utilisée par une **fonction serveur**, jamais par la page elle-même.

Clé	Peut aller dans l'interface ?
Clé **publique** ou « anon » d'un service de données, **avec** des règles d'accès (RLS) bien réglées	**Oui** : elle est faite pour cela, la protection vient des règles

Clé	Peut aller dans l'interface ?
Clé **secrète** ou « service role » d'un service de données	**Jamais** : elle passe outre toutes les règles
Clé d'un service d'**IA**, de **paiement**, d'envoi de **SMS** ou d'**e-mails**	**Jamais** : uniquement côté serveur

Les failles les plus fréquentes du vibe coding

Les experts en sécurité tiennent à jour une liste des failles les plus fréquentes des applications web, l'**OWASP Top 10**. Voici, en mots simples, celles qu'on rencontre le plus avec le code généré par IA :

Faille	En clair	La parade
Contrôle d'accès défaillant	Un utilisateur voit ou modifie les données d'un autre	Règles d'accès dans la base, test des badges (module 6)
Secrets exposés	Clés dans le code de l'interface ou sur GitHub	Variables d'environnement, fonctions serveur, .env ignoré
Saisies non vérifiées	On peut envoyer n'importe quoi dans un formulaire, y compris du code malveillant	Vérifier chaque saisie côté serveur : type, longueur, format
Bibliothèques vulnérables	Des « ingrédients » anciens ont des failles connues	Mises à jour régulières, alertes de sécurité de GitHub
Paquets inventés	L'IA propose parfois une bibliothèque qui n'existe pas, ou au nom piégé	Vérifier que chaque bibliothèque ajoutée est connue et officielle

Les données personnelles

Noms, numéros de téléphone, adresses, montants versés : ce sont des **données à caractère personnel**. En Côte d'Ivoire, la **loi n° 2013-450** encadre leur traitement, et l'**ARTCI** en est l'autorité de protection. Selon la nature de l'application, des **formalités** auprès de l'ARTCI peuvent être nécessaires : renseignez-vous avant de mettre en production une application qui collecte des données personnelles, surtout pour un client.

- **Ne collectez que le nécessaire** : pas de date de naissance si elle ne sert à rien.
- **Informez** les utilisateurs : une page « Confidentialité » qui dit quelles données sont collectées, pourquoi, combien de temps, et comment demander leur suppression.
- **Protégez** : accès limités, mots de passe jamais stockés en clair (les services d'authentification s'en chargent), sauvegardes.
- **Respectez les droits** : accès, rectification, opposition, suppression.

🖱️ PAS À PAS

Étape 1 : chasser les secrets

1. Ouvrez votre application en ligne, **F12** → onglet **Sources** (ou **Réseau**), et cherchez (**Ctrl + F**) des mots comme « key », « secret », « sk_ », « token ».
2. Demandez à l'IA : « Liste toutes les clés et tous les secrets utilisés dans ce projet, et indique pour chacun s'il est exposé dans le code de l'interface. Ne m'affiche aucune valeur. »
3. Pour chaque clé secrète exposée : déplacez son utilisation dans une **fonction serveur** (l'IA sait le faire), mettez la valeur dans les **variables d'environnement**, puis **changez la clé** chez le fournisseur (l'ancienne est compromise).
4. Sur GitHub, vérifiez que le fichier **.env** n'est pas dans le dépôt, et activez les **alertes de sécurité** du dépôt (Settings → Security).
5. Chez chaque fournisseur (IA, SMS, paiement), fixez une **limite de dépenses** et une **alerte**.

Étape 2 : la revue de sécurité par l'IA

> ✍️ **Prompt de revue de sécurité (mode plan)**
>
> « Fais une revue de sécurité complète de cette application, sans rien modifier. Vérifie : les règles d'accès de chaque table pour chaque rôle ; les secrets exposés dans l'interface ; la vérification des saisies côté serveur ; les pages accessibles sans connexion ; les bibliothèques obsolètes ou inconnues ; les données personnelles collectées. Classe les problèmes du plus grave au moins grave, et propose pour chacun la correction la plus simple. »

Corrigez **une** faille à la fois, du plus grave au moins grave, en vérifiant après chaque correction. Puis refaites la revue.

Étape 3 : la qualité

Domaine	Comment vérifier
Téléphone	Mode appareil (F12) et vrais téléphones, dont un petit modèle d'entrée de gamme.
Vitesse	F12 → onglet **Lighthouse** → analyse **Mobile** : visez un bon score en performance. Images optimisées, peu de bibliothèques lourdes.
Accessibilité	Même rapport Lighthouse, partie **Accessibilité** : contrastes suffisants, textes alternatifs des images, boutons assez grands, navigation possible au clavier.
Textes	Orthographe, ton, montants en F CFA, dates au format JJ/MM/AAAA, aucun texte d'exemple oublié.
Erreurs	Console sans erreur rouge sur chaque page ; messages d'erreur compréhensibles pour l'utilisateur (« Le numéro doit contenir 10 chiffres » plutôt que « Error 422 »).

Étape 4 : la liste de contrôle avant livraison

☐ Aucune clé secrète dans l'interface ni sur GitHub ; limites de dépenses réglées chez les fournisseurs.

☐ Test des badges réussi pour chaque rôle (module 6).

☐ Chaque saisie est vérifiée côté serveur.

☐ Revue de sécurité refaite après corrections : plus de problème grave.

☐ Page « Confidentialité » en ligne ; formalités ARTCI vérifiées si des données personnelles sont collectées.

☐ Rapport Lighthouse mobile correct ; testé sur trois téléphones.

☐ Sauvegarde des données prévue (export régulier de la base, ou sauvegarde du service).

☐ Comptes administrateurs : au moins deux, avec double authentification.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
🔑 Donner une clé secrète à l'IA « pour que ça marche »	Elle peut l'écrire dans le code de l'interface.	Demander explicitement une fonction serveur et des variables d'environnement.
🙈 Croire que ce qui n'est pas affiché est caché	Tout le code de l'interface est lisible avec F12.	Les secrets et les règles dans l'arrière-boutique.
📦 Accepter n'importe quelle bibliothèque proposée	Bibliothèque inconnue, obsolète ou piégée.	Vérifier son nom, sa popularité, sa date de mise à jour.
🗃️ Collecter « au cas où »	Plus de données = plus de risques et d'obligations.	Le strict nécessaire.
💳 Pas de limite de dépenses	Une fuite ou un bug coûte des centaines de milliers de francs.	Plafonds et alertes chez chaque fournisseur.

🙋 Les questions qu'on n'ose pas poser

« Mon application est petite. Qui voudrait l'attaquer ? »

Personne en particulier : des **robots** parcourent Internet en permanence à la recherche de clés exposées et de failles connues, sur toutes les applications, petites ou grandes. La taille ne protège pas.

> **« L'IA dit que l'application est « sécurisée ». C'est suffisant ? »**
>
> Non. Demandez-lui de **prouver** : quels tests, quelles règles, quels résultats. Faites vous-même le test des badges et la chasse aux secrets. Pour une application sensible (paiements, santé, nombreuses données), faites appel à un professionnel de la sécurité.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : vitrine ou arrière-boutique ?

Élément	Interface (vitrine) ou serveur (arrière-boutique) ?
La couleur des boutons	
La clé secrète du service de paiement	
La vérification que le montant est un nombre positif	
La clé publique du service de données, avec RLS	
Le calcul du total facturé au client	

🟡 Je m'entraîne

Exercice 2 : la chasse aux secrets

Appliquez l'étape 1 à votre projet. Notez chaque secret trouvé, où il était, et ce que vous avez fait.

🔴 Je me lance

Exercice 3 : la revue complète

Faites la revue de sécurité par l'IA, corrigez les problèmes, passez Lighthouse, et cochez toute la liste de contrôle avant livraison. Rédigez la page « Confidentialité » de votre application.

🧩 CAS PRATIQUE

Après la facture

Reprenez la situation de Yao : clé secrète d'un service d'IA dans le code de l'interface, 680 dollars consommés en une nuit.

1. Que doit faire Yao **dans l'heure** ?
2. Comment corriger durablement l'application ?

3. Que peut-il demander au fournisseur ?
4. Quelles règles ajoute-t-il dans les instructions permanentes de tous ses projets ?

📌 CE QUE JE RETIENS

- Tout le code de l'interface est **lisible** : les clés secrètes vont dans l'**arrière-boutique** (fonctions serveur, variables d'environnement).
- Les failles fréquentes : **accès**, **secrets**, **saisies non vérifiées**, **bibliothèques** obsolètes ou inventées.
- **Limites de dépenses** et alertes chez chaque fournisseur.
- Données personnelles : le **nécessaire**, une page **Confidentialité**, les droits des personnes, la **loi n° 2013-450** et l'**ARTCI**.
- Qualité : téléphone, **Lighthouse** (vitesse, accessibilité), textes, messages d'erreur clairs.
- La **revue de sécurité** et la **liste de contrôle** avant chaque livraison.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, faites la chasse aux secrets et la revue de sécurité de **tous** vos projets en ligne, même les petits essais. Changez toute clé qui a été exposée, même un instant.

💬 Le mot de la fin

Yao a changé sa clé dans l'heure, écrit au fournisseur pour expliquer l'abus, et obtenu un geste commercial sur une partie de la facture. Puis il a ajouté une ligne dans les instructions permanentes de tous ses projets : « Aucune clé secrète dans l'interface. Jamais. Même pour un essai. » Il ne l'a plus jamais enlevée.

📝 Mini-quiz : ai-je bien compris ?

1. Une clé secrète de paiement peut être placée… **a)** dans le code de l'interface · **b)** uniquement côté serveur, dans les variables d'environnement · **c)** dans le README.
2. L'OWASP Top 10 est… **a)** une liste des failles les plus fréquentes des applications web · **b)** un classement de téléphones · **c)** un hébergeur.
3. Pour vérifier la vitesse et l'accessibilité d'une page, j'utilise… **a)** Lighthouse dans les outils de développement · **b)** la Corbeille · **c)** WhatsApp.
4. En matière de données personnelles, la bonne règle est… **a)** tout collecter au cas où · **b)** collecter le strict nécessaire et informer les utilisateurs · **c)** ne rien dire.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 11

Brancher son application au monde

Le maquis qui accepte Wave, Orange Money, MTN MoMo et Moov Money

🎯 **À la fin de ce module, je saurai…**

- comprendre comment une application parle à d'autres services par **API** ;
- intégrer un **paiement Mobile Money** par un agrégateur de paiement ;
- comprendre les **webhooks** : être prévenu quand un paiement est confirmé ;
- envoyer des **e-mails** et des **messages** automatiques ;
- ajouter une fonction d'**intelligence artificielle** dans son application.

Durée conseillée	Matériel	Prérequis
Six heures	Votre projet en ligne, un compte de test chez un agrégateur de paiement	Modules 1 à 10

📖 LA SITUATION

Le rendez-vous non honoré

Chez Koffi Services, un client sur quatre réserve un rendez-vous… et ne vient pas. Le créneau est perdu. Koffi voudrait demander un **acompte de 1 000 F** à la réservation, payable par Mobile Money, et que le client reçoive automatiquement une confirmation, puis un rappel la veille.

Koffi : Au maquis d'à côté, on peut payer avec n'importe quel opérateur. Pourquoi pas sur mon application ?

M. Kouassi : Le maquis n'a pas signé avec chaque opérateur, Koffi : il passe par un intermédiaire qui les accepte tous. Ton application fera pareil. Mais l'argent, c'est sérieux : on va le faire avec méthode.

💡 LE DÉCLIC

L'API, c'est le serveur du maquis

Au maquis, vous ne rentrez pas dans la cuisine. Vous donnez votre commande au **serveur**, dans un format convenu (« deux garbas, un alloco »), il la transmet à la cuisine, et vous rapporte le plat. Une **API** fait la même chose entre deux programmes : votre application envoie une demande dans un format convenu (« crée un paiement de 1 000 F pour ce client »), le service la traite et renvoie une réponse.

L'agrégateur de paiement

Signer un contrat avec chaque opérateur Mobile Money serait long et compliqué. Un **agrégateur de paiement** (en Afrique de l'Ouest, par exemple CinetPay, PayDunya, FedaPay, Paystack…) le fait pour vous : avec **une seule** intégration, vos clients paient par Orange Money, MTN MoMo, Moov Money, Wave ou carte bancaire, selon les moyens proposés par l'agrégateur. Il prend une **commission** par transaction et vous reverse le reste.

> ⚠️ **À vérifier avant de choisir**
>
> Les moyens de paiement acceptés, les **frais**, le délai de **reversement**, les **documents** demandés pour ouvrir un compte marchand (souvent ceux d'une entreprise déclarée), et la qualité de la **documentation** technique. Ces éléments changent : comparez sur les sites officiels au moment de votre projet.

Le webhook : le coup de fil de confirmation

Quand le client paie sur son téléphone, votre application ne le sait pas tout de suite. C'est l'agrégateur qui **appelle** votre application pour la prévenir : « Le paiement n° 123 est confirmé. » Cet appel automatique s'appelle un **webhook**. C'est **lui**, et seulement lui, qui doit faire passer la réservation en « payée », après vérification.

Étape	Qui fait quoi
1	Le client choisit son créneau et clique sur « Payer l'acompte ».
2	Votre **serveur** demande à l'agrégateur de créer un paiement (avec la clé secrète, côté serveur).
3	Le client est envoyé sur la page de paiement de l'agrégateur et valide sur son téléphone.
4	L'agrégateur appelle votre **webhook** : paiement confirmé ou refusé.
5	Votre serveur **vérifie** l'authenticité de l'appel et le montant, puis marque la réservation « payée ».
6	L'application envoie la confirmation au client.

🖱️ **PAS À PAS**

Étape 1 : le paiement en mode test

1. Créez un compte chez l'agrégateur choisi et activez le **mode test** (ou « sandbox ») : de faux paiements, pour essayer sans argent réel.
2. Récupérez les **clés de test** et placez-les dans les **variables d'environnement** (jamais dans le code de l'interface : module 10).
3. Donnez à l'IA la **documentation officielle** de l'agrégateur (lien ou extrait) : elle travaillera beaucoup mieux qu'en devinant.

> ✍️ **Prompt (mode plan d'abord)**
>
> « Intègre le paiement d'un acompte de 1 000 F CFA avec [nom de l'agrégateur], dont voici la documentation : [lien]. Utilise les clés de test des variables d'environnement [NOMS DES VARIABLES]. La création du paiement et la réception du webhook doivent se faire dans des fonctions serveur. Le webhook doit vérifier l'authenticité de l'appel selon la documentation, vérifier le montant, et seulement ensuite passer la réservation en « payée ». Une réservation non payée sous 30 minutes est libérée. Présente-moi d'abord le plan et les cas d'échec que tu vas gérer. »

Étape 2 : tester tous les cas

- ☐ Paiement réussi : la réservation passe en « payée », le client reçoit la confirmation.
- ☐ Paiement refusé ou annulé : la réservation reste en attente, puis est libérée.
- ☐ Le client ferme la page avant de payer : rien n'est marqué payé.
- ☐ Le webhook arrive deux fois : la réservation n'est pas comptée deux fois.
- ☐ Un faux appel au webhook (envoyé par quelqu'un d'autre que l'agrégateur) est refusé.
- ☐ Les montants affichés, payés et enregistrés sont identiques.

Ce n'est qu'après ces tests, et un petit paiement réel de vérification, qu'on passe aux **clés de production**.

Étape 3 : e-mails et messages automatiques

Canal	Comment	À savoir
E-mail	Un service d'envoi d'e-mails transactionnels (Resend, Brevo, SendGrid…), appelé depuis une fonction serveur	Configurer son nom de domaine pour que les e-mails n'arrivent pas en spam
SMS	Un fournisseur de SMS (dont certains agrégateurs africains)	Coût par message : prévoir un budget
WhatsApp	Simple : un lien wa.me prérempli (module 5). Automatique : l'**API WhatsApp Business** de Meta, via un fournisseur	L'API demande une validation, des modèles de messages approuvés, et elle est payante selon les conversations

Pour le rappel « la veille », il faut une **tâche programmée** (on dit aussi « cron ») : une fonction serveur qui s'exécute chaque jour à 18 h et envoie les rappels du lendemain. Demandez-la à l'IA en précisant le fuseau horaire : **Africa/Abidjan**.

Étape 4 : ajouter de l'IA dans son application

Exemple : Tantie Akissi voudrait un bouton « Rédiger la description » qui propose un texte pour chaque nouveau pagne.

1. Créez une clé chez un fournisseur d'IA (Anthropic, OpenAI, Google…) ; certains outils de vibe coding proposent aussi l'IA intégrée.

2. Fixez une **limite de dépenses**.
3. Demandez à l'IA de votre outil : « Ajoute un bouton “Rédiger la description” qui envoie le nom, la catégorie et la couleur du pagne à une fonction serveur, laquelle appelle l'API d'IA avec la clé des variables d'environnement et renvoie une description de 25 mots, ton chaleureux. La trésorière doit pouvoir modifier le texte avant de l'enregistrer. »
4. Testez, puis limitez l'usage (par exemple dix descriptions par jour) pour maîtriser les coûts.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
✅ Marquer « payé » dès que le client revient sur l'application	Le client peut revenir sans avoir payé.	Seul le webhook vérifié fait foi.
🔑 Clés de paiement dans l'interface	Vol, fraude, transactions frauduleuses.	Fonctions serveur et variables d'environnement.
🧪 Passer en production sans tester les cas d'échec	Réservations payées non enregistrées, clients furieux.	La liste des six tests, en mode test.
💸 Oublier les commissions et coûts des messages	La marge disparaît.	Les inclure dans le prix ou dans l'acompte.
📵 Envoyer des messages non sollicités	Clients agacés, compte bloqué par les plateformes.	Seulement des messages utiles, avec l'accord du client.

🙋🏾 Les questions qu'on n'ose pas poser

« Faut-il une entreprise déclarée pour encaisser en ligne ? »

La plupart des agrégateurs demandent des documents d'entreprise pour ouvrir un compte marchand en production. Renseignez-vous sur leurs conditions ; pour un client, c'est en général **son** entreprise qui ouvre le compte, pas vous.

« Et la loi ? »

Les transactions électroniques sont encadrées en Côte d'Ivoire, notamment par la loi n° 2013-546 relative aux transactions électroniques. Affichez clairement les prix, les conditions (acompte remboursable ou non) et l'identité du vendeur.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : remettre le paiement dans l'ordre

Remettez dans l'ordre : « Le webhook confirme le paiement » · « Le client valide sur son téléphone » · « Le serveur crée le paiement chez l'agrégateur » · « La réservation passe en payée » · « Le client clique sur Payer ».

1. .....

2. .....

3. .....

4. .....

5. .....

🟡 Je m'entraîne

Exercice 2 : le paiement de test

Intégrez un paiement en mode test dans votre projet (ou dans une copie de la vitrine de Pagnes Akissi), et passez les six tests de l'étape 2.

🔴 Je me lance

Exercice 3 : le rappel automatique

Ajoutez à votre projet un e-mail de confirmation et une tâche programmée qui envoie un rappel la veille, au bon fuseau horaire.

🧩 CAS PRATIQUE

L'acompte de Koffi

Koffi lance l'acompte de 1 000 F. Après une semaine : 85 réservations payées selon l'agrégateur, mais seulement 79 marquées « payées » dans l'application. Deux clients se plaignent d'avoir payé sans recevoir de confirmation.

1. Quelles sont les causes possibles de l'écart ?
2. Où regarder pour trouver la cause ?
3. Que faire immédiatement pour les clients concernés ?
4. Quelle amélioration durable proposer ?

📌 CE QUE JE RETIENS

- Une **API**, c'est le serveur du maquis : une demande dans un format convenu, une réponse.
- Un **agrégateur** de paiement : une intégration pour Mobile Money et cartes, contre une commission.
- Le **webhook** vérifié est la **seule** preuve de paiement.
- **Mode test** d'abord, six cas testés, puis production.
- Clés en **variables d'environnement**, appels dans des **fonctions serveur**, **limites de dépenses**.
- E-mails, SMS, WhatsApp : utiles, consentis, et dont on connaît le **coût**. Fuseau **Africa/Abidjan**.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, comparez trois agrégateurs de paiement disponibles en Côte d'Ivoire : moyens acceptés, frais, documents demandés, délai de reversement. Présentez votre tableau à votre binôme.

💬 Le mot de la fin

Après l'acompte, le taux de rendez-vous non honorés de Koffi est passé d'un sur quatre à un sur vingt. Les clients reçoivent un message la veille : « Bonjour, nous vous attendons demain à 10 h chez Koffi Services. » Un client a répondu : « Même ma banque ne me rappelle pas comme ça ! »

📝 Mini-quiz : ai-je bien compris ?

1. Un agrégateur de paiement permet… **a)** d'accepter plusieurs moyens de paiement avec une seule intégration · **b)** d'héberger le site · **c)** de créer un logo.
2. La preuve qu'un paiement est confirmé, c'est… **a)** le retour du client sur la page · **b)** le webhook vérifié de l'agrégateur · **c)** un message WhatsApp du client.
3. Avant la production, on teste en… **a)** mode test (sandbox) · **b)** direct avec de vrais clients · **c)** mode avion.
4. Pour un rappel la veille à 18 h, on utilise… **a)** une tâche programmée · **b)** la Corbeille · **c)** le README.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 12

Automatisation et IA avancée : API, Make, Zapier, agents

L'employé qui ne dort jamais

🎯 **À la fin de ce module, je saurai…**

- repérer les **tâches répétitives** qui méritent d'être automatisées ;
- construire un **scénario** avec **Make** ou **Zapier** : déclencheur, actions, filtres ;
- utiliser une **API** dans un scénario, sans programmer ;
- ajouter de l'**IA** dans une automatisation ;
- comprendre les **agents d'IA** et les garder **sous contrôle**.

Durée conseillée	Matériel	Prérequis
Six heures	Un compte Make ou Zapier (offre gratuite), Google Forms et Sheets, Gmail	Modules 1 à 11 ; Tome 2, module 5

📖 LA SITUATION

Les soirées de Mariam

Chaque soir, Mariam passe une heure à la même routine : elle ouvre les demandes de devis reçues par le formulaire du site, les recopie dans le tableau de suivi, envoie à chaque client un e-mail « Nous avons bien reçu votre demande », prévient le commercial concerné sur WhatsApp, et prépare un résumé pour le directeur. Cinq jours par semaine. Deux cent cinquante heures par an.

Mariam : Je ne suis pas devenue assistante de direction pour faire du copier-coller !

M. Kouassi : Alors on va vous embaucher un employé qui ne dort jamais, qui ne se trompe pas en recopiant, et qui ne demande pas d'augmentation. Il faudra juste lui expliquer son travail très précisément… et vérifier qu'il le fait bien.

💡 LE DÉCLIC

Un scénario, c'est une fiche de poste

Une automatisation se décrit comme la fiche de poste d'un employé très obéissant : **quand** il se passe telle chose, **fais** ceci, puis cela, **seulement si** telle condition est vraie.

Élément	Dans la fiche de poste…	Dans Make ou Zapier
Déclencheur	« Quand un client dépose une demande au guichet… »	L'événement qui lance le scénario : nouveau formulaire, nouvel e-mail, nouvelle ligne, paiement reçu, heure fixe
Actions	« … recopie-la dans le registre, envoie un accusé de réception, préviens le commercial. »	Les étapes exécutées l'une après l'autre, dans différentes applications
Filtre	« … seulement si le montant dépasse 500 000 F. »	Une condition qui laisse passer ou arrête la suite
Données transmises	Le nom du client recopié du formulaire vers le registre	Les champs d'une étape réutilisés dans les suivantes

Outil	Points forts	À savoir
Zapier	Très simple, des milliers d'applications reliées	Offre gratuite limitée ; tarif selon le nombre de tâches
Make	Visuel (des bulles reliées), très souple, souvent moins cher pour des scénarios complexes	Un peu plus technique ; tarif selon le nombre d'opérations
n8n	Libre, peut être installé sur son propre serveur	Pour les plus avancés

Les agents d'IA

Un scénario Make suit une fiche de poste **fixe**. Un **agent d'IA** reçoit un **objectif** et décide lui-même des étapes, en utilisant des **outils** (chercher sur Internet, lire des e-mails, remplir un tableau, appeler une API). Par exemple : « Chaque matin, lis les nouvelles demandes de devis, classe-les par urgence, prépare un brouillon de réponse pour chacune, et fais-moi un résumé. »

🔑 **L'agent est un stagiaire autonome : il faut des garde-fous**

- **Validation humaine** avant toute action irréversible ou extérieure : envoyer un e-mail à un client, payer, supprimer, publier.
- **Droits minimaux** : l'agent n'accède qu'à ce dont il a besoin (lecture seule quand c'est possible).
- **Limites** de dépenses et de nombre d'actions.
- **Journal** de tout ce qu'il fait, relu régulièrement.
- **Méfiance** envers le contenu qu'il lit : un e-mail ou une page web peuvent contenir des instructions piégées destinées à le détourner.

PAS À PAS

Étape 1 : repérer ce qui mérite d'être automatisé

Une tâche est une bonne candidate si elle est **répétitive**, **régulière**, **à règles claires**, et qu'une erreur n'est pas dramatique. Listez vos tâches de la semaine et estimez pour chacune : temps par fois × nombre de fois par mois.

Étape 2 : le scénario de Mariam avec Make

1. Créez un compte sur **make.com** → **Create a new scenario**.
2. **Déclencheur** : cliquez sur le **+**, choisissez **Google Forms** (ou **Google Sheets** → **Watch New Rows** sur la feuille des réponses du formulaire), connectez votre compte Google, choisissez le formulaire des devis.
3. **Action 1** : **Google Sheets** → **Add a Row** dans le tableau de suivi. Glissez chaque champ du formulaire (nom, téléphone, besoin, montant estimé) dans la bonne colonne ; ajoutez la date et le statut « Nouveau ».
4. **Action 2** : **Gmail** → **Send an Email** au client : objet « Nous avons bien reçu votre demande », message personnalisé avec son prénom.
5. **Filtre** : entre deux bulles, cliquez sur le lien → **Set up a filter** : « Montant estimé supérieur à 500 000 ».
6. **Action 3** (après le filtre) : prévenir le directeur par e-mail, ou par un message dans l'outil de discussion de l'équipe.
7. Cliquez sur **Run once** et remplissez le formulaire avec une fausse demande : suivez les bulles qui s'allument. Vérifiez la ligne ajoutée et l'e-mail reçu.
8. Activez le scénario (**ON**) et choisissez sa fréquence.

ILLUSTRATION

Capture d'écran d'un scénario Make : une bulle Google Forms reliée à Google Sheets (Add a Row), puis à Gmail (Send an Email), puis un filtre « Montant > 500 000 » vers une bulle « Prévenir le directeur ».

Étape 3 : ajouter l'IA au scénario

1. Entre le déclencheur et l'enregistrement, ajoutez une bulle d'IA (module OpenAI, Anthropic Claude ou Google Gemini, selon ceux proposés, avec votre clé).
2. Instruction : « Voici une demande de devis : [besoin]. Classe-la dans une seule de ces catégories : Impression, Saisie, Conception graphique, Autre. Donne aussi une urgence : Haute, Normale, Basse. Réponds uniquement au format : catégorie;urgence. »
3. Utilisez la réponse dans les colonnes « Catégorie » et « Urgence » du tableau de suivi.
4. Pendant deux semaines, **vérifiez** chaque classement. Corrigez l'instruction si l'IA se trompe souvent.

Étape 4 : appeler une API sans programmer

Quand une application n'a pas de bulle toute prête, la bulle **HTTP** (« Make a request ») permet d'appeler directement son API, à partir de sa **documentation** : une adresse, une méthode (GET pour lire, POST pour envoyer), éventuellement une clé. Demandez à un assistant d'IA : « Voici la documentation de cette API : [extrait]. Comment configurer la bulle HTTP de Make pour [ce que je veux] ? »

Étape 5 : un premier agent, sous contrôle

1. Commencez par un agent qui **prépare** sans **agir** : il lit les demandes du jour et rédige des **brouillons** de réponse dans Gmail, que Mariam relit et envoie elle-même.
2. Donnez-lui une instruction écrite précise : son rôle, ses sources, ce qu'il a le droit de faire, ce qu'il ne doit **jamais** faire, le format de son compte rendu.
3. Faites-le tourner en parallèle de la méthode manuelle pendant une ou deux semaines, et comparez.
4. N'élargissez ses droits qu'après avoir constaté sa fiabilité, et gardez toujours la validation humaine pour ce qui part chez un client.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
🤖 Automatiser un processus mal défini	On automatise le désordre, plus vite.	Écrire d'abord la fiche de poste, à la main.
🔁 Un scénario qui se déclenche lui-même en boucle	Des centaines d'e-mails envoyés, quota épuisé.	Tester avec Run once ; filtres ; limites.
📧 Laisser l'IA écrire directement aux clients	Une erreur ou une invention part sous votre nom.	Brouillons validés par un humain.
🔑 Donner à un agent tous les accès	Une erreur ou un détournement peut tout toucher.	Droits minimaux, lecture seule si possible.
🙈 Ne plus regarder un scénario qui « marche »	Une connexion expire, rien ne se passe pendant des semaines.	Alertes d'erreur activées, contrôle hebdomadaire.

🙋 Les questions qu'on n'ose pas poser

> **« Combien ça coûte ? »**
>
> Les offres gratuites permettent d'apprendre et de faire tourner de petits scénarios. Au-delà, Make et Zapier facturent selon le nombre d'opérations ou de tâches par mois, et les modules d'IA consomment des crédits chez le fournisseur d'IA. Calculez : le coût mensuel est-il inférieur au temps gagné ?

> **« Automatiser, c'est supprimer des emplois ? »**
>
> Bien utilisée, l'automatisation supprime surtout des **tâches** ennuyeuses et libère du temps pour ce qui demande du jugement, de la relation et de la créativité. Mariam n'a pas perdu son poste : elle a gagné cinq heures par semaine pour des missions plus importantes.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : déclencheur, action ou filtre ?

Élément	Rôle
Chaque lundi à 8 h	
Envoyer un SMS au client	
Seulement si le client est à Abidjan	
Nouvelle réponse au formulaire	
Ajouter une ligne dans Google Sheets	

🟡 Je m'entraîne

Exercice 2 : le scénario de Mariam

Reproduisez le scénario complet de l'étape 2 avec votre propre formulaire, puis ajoutez la bulle d'IA de l'étape 3. Testez avec cinq fausses demandes variées.

🔴 Je me lance

Exercice 3 : ma fiche de poste

Choisissez une vraie tâche répétitive de votre vie ou de votre travail. Écrivez sa fiche de poste (déclencheur, actions, filtres), estimez le temps gagné par mois, puis automatisez-la.

🧩 CAS PRATIQUE

L'agent de la coopérative

Une coopérative de productrices de vivriers de Daloa reçoit chaque jour des commandes de restaurants d'Abidjan par WhatsApp et par e-mail. La présidente voudrait « un agent IA qui gère tout : prendre les commandes, confirmer aux clients, réserver le transport et encaisser ».

1. Qu'est-ce qui peut être automatisé par un **scénario fixe** ? Qu'est-ce qui demanderait un **agent** ?
2. Quelles actions doivent **toujours** garder une validation humaine ?

3. Proposez une mise en place en trois étapes, de la plus prudente à la plus autonome.
4. Quels garde-fous mettez-vous en place ?

📌 CE QUE JE RETIENS

- Un scénario = une **fiche de poste** : **déclencheur**, **actions**, **filtres**, données transmises.
- **Zapier** (simple), **Make** (visuel et souple), **n8n** (libre). On teste avec **Run once**.
- La bulle **HTTP** appelle n'importe quelle **API** à partir de sa documentation.
- L'IA dans un scénario : instruction précise, format de réponse imposé, **vérification** au début.
- Un **agent** reçoit un objectif et choisit ses étapes : **validation humaine**, **droits minimaux**, **limites**, **journal**.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, automatisez une tâche répétitive réelle et mesurez le temps gagné. Activez les alertes d'erreur du scénario.

💬 Le mot de la fin

Trois semaines plus tard, le directeur a demandé à Mariam pourquoi elle partait maintenant à 17 h 30. Elle lui a montré le scénario, les bulles qui s'allumaient, le tableau qui se remplissait tout seul, et le résumé du matin préparé par l'agent. Il a réfléchi, puis il a dit : « Vous pourriez faire la même chose pour la comptabilité ? »

📝 Mini-quiz : ai-je bien compris ?

1. Ce qui lance un scénario s'appelle… **a)** le déclencheur · **b)** le filtre · **c)** la clé.
2. Un filtre sert à… **a)** laisser passer la suite seulement si une condition est vraie · **b)** colorer les cellules · **c)** héberger le site.
3. Un agent d'IA… **a)** suit une liste d'étapes fixe · **b)** reçoit un objectif et choisit lui-même ses étapes avec des outils · **c)** est un antivirus.
4. Avant qu'un agent envoie un e-mail à un client, il faut… **a)** une validation humaine · **b)** rien · **c)** redémarrer l'ordinateur.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

MODULE 13

Vivre du vibe coding

De l'atelier à l'entreprise

🎯 **À la fin de ce module, je saurai…**

- définir des **offres** claires adaptées au marché ivoirien ;
- **chiffrer** une prestation et rédiger un **devis** ;
- protéger le client et soi-même avec un **contrat** simple ;
- proposer une **maintenance** et des revenus récurrents ;
- construire un **portfolio** et trouver ses **premiers clients**.

Durée conseillée	Matériel	Prérequis
Quatre heures	Vos projets réalisés, Excel ou Google Sheets	Modules 1 à 12

📖 LA SITUATION

Le premier devis de Yao

Le pharmacien du quartier a vu l'application de la tontine. Il veut une application pour que ses clients réservent leurs médicaments et soient prévenus quand ils arrivent. Il demande : « Combien ? »

Yao, pris de court, répond : « 50 000 F, ça va ? » Le pharmacien accepte immédiatement. Trois semaines plus tard, Yao a passé quatre-vingts heures sur le projet, payé 45 000 F d'abonnements, le pharmacien demande des modifications chaque jour, et il veut maintenant que Yao vienne réparer son imprimante « puisqu'il est informaticien ».

> **Yao :** J'ai travaillé pour moins que rien, et je ne sais même pas quand ça va s'arrêter !
>
> **M. Kouassi :** Tu as fait le travail d'un professionnel, Yao, avec la gestion d'un amateur. Le talent ne suffit pas. Il faut une offre, un prix calculé, un devis, un contrat, et le courage de dire non. On va voir tout ça.

💡 LE DÉCLIC

Le couturier qui affiche ses prix

Un bon couturier n'invente pas son prix à la tête du client. Il sait combien coûtent le tissu et les fournitures, combien d'heures prend chaque modèle, et ce que vaut son savoir-faire. Il affiche ses modèles, il prend les mesures, il annonce un prix et une date, il demande une avance. Et une retouche après la livraison, c'est payant, sauf défaut de sa part.

Des offres claires

Offre	Contenu	Pour qui
Vitrine Essentielle	Site vitrine de 3 à 5 pages, responsive, commande WhatsApp, référencement de base, mise en ligne avec nom de domaine	Commerçants, artisans, églises, associations
Application de gestion	Comptes, base de données, tableau de bord, 3 à 6 écrans	Tontines, petites écoles, cabinets, PME
Automatisation	Un à trois scénarios Make ou Zapier, avec formation	Bureaux, services administratifs
Maintenance	Mises à jour, sauvegardes, petites corrections, surveillance, un nombre d'heures de modifications par mois	Tous les clients, en abonnement mensuel

Le calcul du prix

Un prix se construit, il ne se devine pas :

Élément	Exemple pour une application de gestion
Temps estimé × votre taux horaire	40 heures (cahier des charges, construction, tests, sécurité, formation) × votre taux
Coûts des outils pendant le projet	Abonnements de l'outil de vibe coding, crédits d'IA
Coûts récurrents du client	Hébergement, base de données, nom de domaine, envois de SMS : à faire payer par le client ou à inclure dans la maintenance
Marge d'imprévus	15 à 30 % du temps : il y en a toujours
Valeur pour le client	Combien l'application lui fait gagner ou économiser ? Un outil qui évite 200 000 F de pertes par mois vaut plus que son temps de fabrication

> 🔑 **Votre taux horaire**
>
> Partez de ce que vous voulez gagner par mois, ajoutez vos charges (ordinateur, Internet, abonnements, transport, cotisations), et divisez par les heures **réellement facturables** (souvent la moitié de votre temps de travail : le reste part en prospection, formation, administration). Comparez ensuite avec les prix pratiqués autour de vous, et ajustez.

PAS À PAS

Étape 1 : le devis

Rubrique	Contenu
En-tête	Votre nom ou entreprise, contacts, numéro de devis, date, durée de validité (30 jours).
Client	Nom, entreprise, contacts.
Objet	Le besoin en une phrase, et une référence au cahier des charges joint.
Détail	Chaque lot avec son prix : cahier des charges, construction, tests et sécurité, mise en ligne, formation.
Ce qui est inclus et exclu	Nombre de séries de retouches incluses (par exemple deux) ; ce qui est hors projet (contenus, photos, matériel, dépannage).
Coûts récurrents	Hébergement, domaine, services : montants estimés, qui paie.
Délais	Date de livraison, à condition de recevoir les contenus du client à temps.
Paiement	Par exemple : 40 % à la commande, 40 % à la livraison de la version de test, 20 % à la mise en ligne.
Signature	« Bon pour accord », date, signature du client.

Faites-le dans Word ou Excel avec un modèle propre (Tome 2), exporté en PDF.

Étape 2 : le contrat simple

Pour les projets importants, ajoutez au devis des **conditions** claires, et faites-les relire par un professionnel du droit si les montants sont élevés :

- **Propriété** : le code, le dépôt GitHub, le nom de domaine et les comptes des services appartiennent au client **après paiement complet**. Vous pouvez montrer le projet dans votre portfolio, sauf refus du client.
- **Données** : le client est responsable de ses données et de ses formalités (dont celles prévues par la loi n° 2013-450) ; vous vous engagez à les protéger et à ne pas les utiliser.
- **Modifications** : toute demande hors cahier des charges fait l'objet d'un devis complémentaire.
- **Garantie** : correction gratuite des défauts signalés dans les 30 jours suivant la mise en ligne.
- **Maintenance** : contrat séparé, mensuel, avec un contenu précis.

Étape 3 : la maintenance, le revenu qui revient

Une application vivante a besoin d'entretien : mises à jour de sécurité, sauvegardes vérifiées, petites corrections, renouvellement du domaine. Proposez un **forfait mensuel** (par exemple deux heures de modifications incluses, surveillance, sauvegardes, réponse sous 48 heures). Dix clients en maintenance, c'est un revenu stable qui finance les mois creux.

Étape 4 : le portfolio

1. Créez votre propre site vitrine (vous savez le faire !) : vos offres, trois à six réalisations, des témoignages, un bouton WhatsApp.
2. Pour chaque réalisation : le problème du client, ce que vous avez construit, le résultat chiffré (« 114 rendez-vous en ligne le premier mois »), une capture.
3. Demandez à chaque client satisfait un **témoignage** de deux phrases, avec son accord pour le publier.
4. Reliez-le à votre profil **LinkedIn** (Tome 2, module 9).

Étape 5 : trouver ses premiers clients

Où	Comment
Votre entourage	Commerçants, églises, associations, écoles : commencez par des projets simples, à prix d'ami mais **jamais gratuitement sans contrepartie** (témoignage, droit de montrer).
Le bouche-à-oreille	Chaque client satisfait en amène d'autres : soignez la livraison et le suivi.
LinkedIn et WhatsApp	Montrez vos réalisations, expliquez ce que vous résolvez, pas la technique.
Les plateformes de missions	Plateformes internationales et africaines de travail indépendant : concurrence forte, mais bonne école.
Les produits numériques	Vendez des modèles prêts à l'emploi (vitrine de restaurant, application de tontine, pack d'automatisations) sur une plateforme de vente de produits numériques adaptée aux créateurs africains, comme Chariow.

Étape 6 : continuer d'apprendre

- Les outils changent tous les mois : consacrez une heure par semaine à la **veille** (nouveautés de votre outil, sécurité).
- Apprendre les **bases de la programmation** (JavaScript, SQL) vous rendra bien plus solide face aux projets complexes.
- Des **certifications** reconnues (cloud, sécurité, gestion de projet) rassurent les entreprises.
- Rejoignez des **communautés** de créateurs et de développeurs, en ligne et à Abidjan.

⚠️ LES PIÈGES À ÉVITER

Le piège	Pourquoi c'est dangereux	Le bon réflexe
🎲 Donner un prix au hasard	Projet à perte, découragement.	Le calcul : temps, coûts, imprévus, valeur.

Le piège	Pourquoi c'est dangereux	Le bon réflexe
♾️ Pas de limite aux retouches	Le projet ne finit jamais.	Nombre de séries de retouches dans le devis ; le reste en complément.
💳 Payer les abonnements du client avec son propre compte	Dépendance, litiges, coûts cachés.	Comptes au nom du client, ou refacturation claire.
🤝 Commencer sans acompte ni devis signé	Travail non payé, client qui disparaît.	Devis signé et acompte avant de commencer.
🧰 Accepter tout ce qu'on vous demande (« répare aussi l'imprimante »)	Vous devenez l'informaticien gratuit de tout le quartier.	Une offre claire, et le courage de dire non ou de facturer.

🙋🏾 Les questions qu'on n'ose pas poser

« Je ne suis pas programmeur. Est-ce honnête de vendre des applications faites avec l'IA ? »

Oui, si vous êtes **transparent** sur votre méthode et **responsable** de la qualité : cahier des charges, tests, sécurité, maintenance. Le client paie un résultat qui fonctionne et un accompagnement, pas des lignes de code tapées à la main. Ne promettez jamais ce que vous ne savez pas faire : pour un projet trop complexe ou trop sensible, orientez vers des professionnels ou associez-vous à eux.

« Dois-je créer une entreprise ? »

Pour facturer régulièrement, ouvrir des comptes marchands ou travailler avec des entreprises, une structure déclarée devient vite nécessaire. Renseignez-vous auprès des services publics d'appui à la création d'entreprise et d'un comptable sur le statut adapté à votre situation.

✍️ EXERCICES

🟢 Je m'échauffe

Exercice 1 : mon taux horaire

Ce que je veux gagner par mois : .....

Mes charges mensuelles : .....

Mes heures facturables par mois : .....

Mon taux horaire = (gains + charges) ÷ heures : .....

🟡 Je m'entraîne

Exercice 2 : le devis du pharmacien

Refaites le devis du pharmacien de la situation de départ, avec toutes les rubriques, le calcul détaillé et les conditions. Comparez avec les 50 000 F de Yao.

🔴 Je me lance

Exercice 3 : mon portfolio

Construisez et mettez en ligne votre site portfolio, avec vos offres et au moins trois réalisations (dont vos projets de ce livre). Partagez-le sur LinkedIn.

🧩 CAS PRATIQUE

Repartir sur de bonnes bases avec le pharmacien

Yao doit renégocier avec le pharmacien, avec qui il a déjà accepté 50 000 F et passé quatre-vingts heures.

1. Que peut-il proposer pour que la relation reste bonne et devienne rentable ?
2. Que répondre à la demande de réparer l'imprimante ?
3. Quelle offre de maintenance lui proposer ?
4. Quelles leçons écrire dans son journal de bord pour les prochains clients ?

📌 CE QUE JE RETIENS

- Des **offres claires** : vitrine, application de gestion, automatisation, **maintenance**.
- Le prix se **calcule** : temps × taux, coûts des outils, coûts récurrents, imprévus, valeur pour le client.
- Un **devis** signé, un **acompte**, des retouches **limitées**, des exclusions écrites.
- Un **contrat** simple : propriété, données, modifications, garantie, maintenance.
- La **maintenance** mensuelle : le revenu qui revient.
- Un **portfolio** avec résultats chiffrés et témoignages ; la transparence sur la méthode.

🏆 MON DÉFI DE LA SEMAINE

Cette semaine, rédigez vos trois offres avec leurs prix de départ, et présentez-les à deux clients potentiels de votre entourage.

> 💬 **Le mot de la fin**
>
> Yao a revu le pharmacien avec un document propre : la première version livrée était comprise dans les 50 000 F, la suite ferait l'objet d'un devis, et un forfait de maintenance mensuel était proposé. Pour l'imprimante, il lui a donné le numéro d'un technicien. Le pharmacien a signé. Et le mois suivant, il a recommandé Yao à trois confrères.

📝 Mini-quiz : ai-je bien compris ?

1. Un prix bien construit tient compte… **a)** seulement du temps · **b)** du temps, des coûts, des imprévus et de la valeur pour le client · **c)** de l'humeur du jour.
2. Avant de commencer un projet, il faut… **a)** un devis signé et un acompte · **b)** seulement une poignée de main · **c)** finir le projet.
3. Le code et le nom de domaine appartiennent en général au client… **a)** jamais · **b)** après paiement complet, si le contrat le prévoit · **c)** avant même de commencer.
4. Les revenus récurrents viennent surtout… **a)** de la maintenance mensuelle · **b)** des réductions · **c)** du hasard.

Vérifiez vos réponses avec votre formateur : les corrigés figurent dans le manuel du formateur.

★ PROJET FINAL

Une application complète pour un vrai client

De l'entretien avec le client à la livraison, comme un professionnel

La mission

Trouvez un **vrai client** : une coopérative, une école, une église, une association, un commerce, un cabinet. Résolvez un **vrai problème** avec une application web complète, et livrez-la comme un professionnel. Si vous n'avez pas de client, prenez le sujet de référence ci-dessous.

> 📋 **Le sujet de référence : la coopérative de Daloa**
>
> La **Coopérative des femmes vivrières de Daloa** vend banane plantain, manioc, piment et légumes à des restaurants d'Abidjan. Les commandes arrivent par appels et messages WhatsApp, se perdent, et les paiements sont suivis dans un cahier.
>
> La présidente veut : un **catalogue** en ligne avec les prix de la semaine ; une **commande** en ligne pour les restaurants (avec compte) ; un **tableau de bord** pour la coopérative (commandes du jour, à livrer, payées) ; un **acompte** par Mobile Money ; une **confirmation** automatique et un **résumé** quotidien des commandes envoyé à la responsable logistique.

Les étapes et les livrables

N°	Étape	Livrable	Modules
1	Entretien client et cahier des charges	Notes d'entretien, cahier des charges d'une page validé, croquis des écrans, liste des « et si… ? »	2
2	Devis et accord	Devis signé (même symbolique), conditions de propriété et de données	13
3	Premier prompt et plan	Premier prompt complet, instructions permanentes du projet	4
4	Construction par étapes	Application avec comptes, rôles, données, règles d'accès dans la base	5, 6
5	Paiement et messages	Acompte en mode test puis production, webhook vérifié, confirmation automatique	11
6	Automatisation	Scénario du résumé quotidien (Make ou Zapier), avec alertes d'erreur	12
7	Sauvegarde	Dépôt GitHub privé, commits clairs, README	8
8	Sécurité et qualité	Revue de sécurité, test des badges, Lighthouse, page Confidentialité, liste de contrôle complète	10

N°	Étape	Livrable	Modules
9	Mise en ligne	Application en production sur un nom de domaine, aperçu séparé, Rollback testé	9
10	Livraison	Formation du client (une page de guide), liste des accès et coûts récurrents, proposition de maintenance	13

La soutenance

En vingt minutes, devant un jury (et si possible le client) : le problème, la démarche, une **démonstration en direct** sur téléphone (une commande, un paiement test, le tableau de bord, le résumé automatique), la sécurité, ce que l'IA a fait et ce que **vous** avez décidé et vérifié, les limites et la suite. Le client, s'il est présent, dit en deux phrases ce que l'application change pour lui.

La grille d'évaluation

Critère	Points	Moi	Jury
Cahier des charges clair, validé par le client, cas particuliers traités	3		
Application fonctionnelle, simple, adaptée au téléphone	4		
Sécurité : règles d'accès, secrets protégés, revue faite, liste de contrôle	4		
Paiement et automatisation fiables, cas d'échec gérés	3		
GitHub, mise en ligne, nom de domaine, Rollback	2		
Livraison professionnelle : devis, guide, accès, maintenance	2		
Soutenance : clarté, honnêteté sur l'usage de l'IA	2		
Total	**20**		

> 🎉 **Félicitations**
>
> Avec 14 points ou plus, vous êtes devenu ce que ce livre promettait : un **créateur numérique**, capable de transformer un besoin réel en un outil qui fonctionne, de le livrer et de l'entretenir. Du Tome 1 au Tome 3, vous êtes passé de l'illettrisme numérique à l'auteur. Transmettez maintenant ce que vous avez appris : à votre tour, apprenez à quelqu'un à apprivoiser l'ordinateur.

Annexe A · Glossaire

Tous les mots techniques du livre, expliqués simplement. Le numéro indique le module où le mot est présenté.

Mot	En mots simples	Module
Accessibilité	Le fait qu'une application soit utilisable par tous, y compris les personnes malvoyantes ou ayant d'autres handicaps.	10
Acompte	La partie du prix payée à la commande.	13
Agent d'IA	Une IA qui reçoit un objectif et enchaîne elle-même des actions avec des outils.	12
Agent de code	Une IA capable de travailler sur tout un projet : lire, modifier, tester, lancer des commandes.	1
Agrégateur de paiement	Un intermédiaire qui permet d'accepter plusieurs moyens de paiement avec une seule intégration.	11
Algorithme	Une suite d'étapes précises pour résoudre un problème : une recette.	2
Aperçu (preview)	La version de l'application qu'on peut essayer pendant sa construction.	1
API	Une interface qui permet à deux programmes d'échanger des demandes et des réponses.	3
API WhatsApp Business	L'interface payante de Meta pour envoyer des messages WhatsApp automatiques.	11
Authentification	La vérification de l'identité d'un utilisateur : inscription, connexion.	3
Automatisation	L'exécution automatique d'une suite de tâches selon des règles.	12
Autorisation	Les droits d'un utilisateur une fois connecté : ce qu'il peut voir et faire.	6
Back-end (serveur)	La partie invisible qui traite les demandes, applique les règles et accède aux données.	3
Base de données	L'endroit où les informations d'une application sont rangées en tables.	3
Bibliothèque de prompts	Une collection personnelle de demandes efficaces, réutilisables.	4
Boucle (répétition)	Une action répétée pour chaque élément d'une liste.	2
Branche	Une version parallèle du code, pour essayer sans toucher à la version principale.	8

Mot	En mots simples	Module
Bug	Un défaut qui empêche une application de fonctionner comme prévu.	7
Bureau d'enregistrement	L'entreprise auprès de laquelle on achète et renouvelle un nom de domaine.	9
Cahier des charges	Le document qui décrit ce que l'application doit faire, pour qui et selon quelles règles.	2
Carnet de bugs	Le tableau personnel des problèmes rencontrés, de leurs causes et de leurs solutions.	7
Clé secrète	Un code qui donne accès à un service payant ou sensible ; elle ne doit jamais être visible.	10
Code	Les instructions écrites dans un langage que l'ordinateur sait exécuter.	1
Code d'état HTTP	Un nombre qui résume la réponse d'un serveur : 401, 403, 404, 500…	7
Commit	Un enregistrement daté des modifications, avec un message.	8
Condition	Une règle du type « si… alors… sinon… ».	2
Console	L'onglet des outils de développement qui affiche les messages et les erreurs d'une page.	7
Constructeur d'applications par IA	Un service en ligne qui crée une application à partir de descriptions en langage courant.	1
Déclencheur	L'événement qui lance une automatisation.	12
Déploiement	La mise en ligne d'une version d'une application.	9
Dépôt (repository)	Le dossier d'un projet avec tout son code et son historique.	8
Devis	Le document qui détaille une prestation, son prix et ses conditions, à faire signer avant de commencer.	13
Données de test	Des données fictives mais réalistes, utilisées pendant la construction.	6
E-mail transactionnel	Un e-mail envoyé automatiquement après une action : confirmation, rappel, reçu.	11
Enregistrement DNS	Une ligne de réglage qui indique où trouver un site pour un nom de domaine (A, CNAME…).	9
Filtre	Une condition qui laisse passer ou arrête la suite d'un scénario.	12
Fonction serveur	Un petit programme exécuté côté serveur, qui peut utiliser des secrets sans les exposer.	10

Mot	En mots simples	Module
Front-end (interface)	La partie visible d'une application : pages, boutons, formulaires.	3
Git	Le système qui enregistre l'historique des modifications du code.	8
GitHub	Le site en ligne qui héberge les dépôts Git et facilite le travail à plusieurs.	8
Google Business Profile	La fiche gratuite d'un établissement sur Google Maps et la recherche Google.	5
Hébergeur	L'entreprise qui fait fonctionner une application sur ses serveurs, accessible par Internet.	3
HTML, CSS, JavaScript	Les trois langages des pages web : structure, apparence, comportement.	3
Instructions de projet	Des consignes permanentes que l'IA relit à chaque demande.	4
Journaux (logs)	Les traces écrites de ce qui se passe sur un serveur ou un service.	7
Lien wa.me	Un lien qui ouvre une discussion WhatsApp avec un numéro, avec un message prérempli.	5
Lighthouse	L'outil intégré à Chrome qui évalue la performance, l'accessibilité et les bonnes pratiques d'une page.	10
Limite de dépenses	Un plafond fixé chez un fournisseur pour éviter les factures surprises.	10
Maintenance	L'entretien régulier d'une application après sa livraison, souvent en abonnement mensuel.	13
Maître d'ouvrage	Celui qui définit le besoin et contrôle le résultat, sans réaliser lui-même les travaux.	1
Make / Zapier / n8n	Des outils pour créer des automatisations entre applications, sans programmer.	12
Maquette	Un dessin des écrans, sur papier ou à l'ordinateur, avant la construction.	2
Méthode SAVE	Symptôme, Action, Voulu, Erreur : pour décrire un problème à l'IA.	7
Mode plan	Un mode de discussion où l'IA réfléchit et propose sans modifier le code.	4
Mode test (sandbox)	Un environnement où l'on fait de faux paiements pour essayer sans argent réel.	11
MVP (produit minimum viable)	La première version, réduite à l'essentiel, pour être utile et recueillir des avis.	2

Mot	En mots simples	Module
NaN	« Not a Number » : le résultat d'un calcul fait avec une valeur qui n'est pas un nombre.	7
Navigation privée	Un mode du navigateur sans mémoire de connexion, utile pour tester un autre compte.	6
Nom de domaine	L'adresse lisible d'un site, comme pagnesakissi.ci.	3
Outils de développement	Les outils intégrés au navigateur (F12) pour inspecter une page et lire les erreurs.	3
OWASP Top 10	La liste de référence des failles de sécurité les plus fréquentes des applications web.	10
Portfolio	La présentation de ses réalisations pour convaincre de futurs clients.	13
Premier prompt	La demande initiale qui décrit toute l'application, à partir du cahier des charges.	4
Préproduction (aperçu)	Une version de test, séparée de la production.	9
Production	La version d'une application utilisée par les vrais utilisateurs.	9
Prompt	La demande écrite adressée à l'IA.	4
Pull Request	Une demande de fusion d'une branche, qui montre les modifications avant de les accepter.	8
README	Le fichier qui présente et explique un projet.	8
Recette	La vérification complète d'une application avant sa livraison au client.	5
Récit utilisateur	Une phrase qui décrit un besoin : En tant que…, je veux…, afin de…	2
Référencement (SEO)	L'ensemble des réglages qui aident un site à être trouvé sur les moteurs de recherche.	5
Requête HTTP	Une demande envoyée à une API à une adresse donnée, pour lire (GET) ou envoyer (POST) des données.	12
Responsive	Qui s'adapte automatiquement à la taille de l'écran : téléphone, tablette, ordinateur.	5
Retouche	Une demande de modification précise, une à la fois.	4
Revenu récurrent	Un revenu qui revient chaque mois, comme un abonnement de maintenance.	13
RLS (sécurité au niveau des lignes)	Des règles dans la base de données qui limitent les lignes qu'un utilisateur peut lire ou modifier.	6

Mot	En mots simples	Module
Rôle	Une catégorie d'utilisateurs ayant les mêmes droits : administrateur, membre…	6
Rollback	Le retour rapide à une version précédente d'une application en ligne.	9
Scénario (Zap)	Une automatisation dans Make (scénario) ou Zapier (Zap).	12
Site vitrine	Un site qui présente une activité et ses produits, sans comptes ni paiement.	5
Supabase	Un service qui fournit base de données, authentification et stockage de fichiers, souvent utilisé par les outils de vibe coding.	6
Table	Un ensemble de données de même nature, en lignes et colonnes, dans une base de données.	3
Tâche programmée (cron)	Une action exécutée automatiquement à une heure fixe.	11
Taux horaire	Le prix d'une heure de travail, calculé à partir des revenus visés, des charges et des heures facturables.	13
Validation des saisies	La vérification de ce que l'utilisateur envoie : type, longueur, format.	10
Validation humaine	L'étape où une personne vérifie et approuve une action avant qu'elle soit exécutée.	12
Variables d'environnement	Des réglages, souvent secrets, fournis à l'application par l'hébergeur plutôt qu'écrits dans le code.	9
Vercel / Netlify	Des hébergeurs qui mettent en ligne des applications web directement depuis GitHub.	9
Vibe coding	Créer des applications en décrivant ce qu'on veut à une IA, qui écrit le code.	1
Webhook	Un appel automatique d'un service vers votre application pour la prévenir d'un événement.	11

Annexe B · La bibliothèque de prompts

Vingt prompts éprouvés, à adapter à vos projets. Remplacez les crochets [ ] par vos informations. Ajoutez vos propres trouvailles dans votre journal de bord.

Concevoir

1. Organiser un cahier des charges

« Voici mes notes d'entretien avec un client : [notes]. Organise-les en cahier des charges d'une page : Objectif, Utilisateurs et rôles, Écrans, Actions, Données, Règles, Contraintes, Hors première version. Puis liste les cas particuliers que j'ai oubliés, sous forme de questions “Et si… ?”. »

2. Les récits utilisateurs

« À partir de ce cahier des charges, écris les récits utilisateurs au format “En tant que…, je veux…, afin de…”, classés par rôle et par priorité. »

3. Le modèle de données

« Ne code rien. Propose les tables de données pour cette application, avec leurs colonnes, leurs types et leurs relations, puis les règles d'accès de chaque rôle sur chaque table. »

Construire

4. Le premier prompt

« But : [ ]. Utilisateurs et rôles : [ ]. Écrans : [ ]. Données : [ ]. Règles : [ ]. Style : [ ]. Contraintes : téléphone d'abord, français, F CFA, dates JJ/MM/AAAA, aucune clé secrète dans l'interface. Présente d'abord ton plan et pose-moi tes questions, puis construis seulement le premier écran. »

5. Ajouter une fonction

« Sur la page [ ], ajoute [fonction] qui permet à [rôle] de [action]. Quand [événement], il doit se passer [résultat]. Gère les cas : [cas particuliers]. Ne modifie rien d'autre. »

6. Retouche visuelle

« Sur téléphone, dans [zone], [problème précis, avec capture jointe]. Je voudrais [résultat]. Ne touche pas au reste de la page. »

7. Données de test

« Crée des données de test réalistes : [nombre] utilisateurs avec des noms ivoiriens, [données], dont des cas difficiles : [retards, doublons, valeurs vides, montants élevés]. »

Comprendre et vérifier

8. Expliquer le code

« Explique-moi ce fichier comme à quelqu'un qui ne programme pas : à quoi il sert, comment il est organisé, ce qui casserait si je le modifiais. Utilise une comparaison avec une boutique. »

9. Le plan de l'application

« Fais-moi le plan de l'application : pages, tables, fonctions serveur, services externes, variables d'environnement (noms seulement), et comment tout est relié. »

10. Les cas d'échec

« Relis [fonction] et liste tous les cas où elle pourrait mal se comporter : données vides, connexion lente, double clic, utilisateur non autorisé. Propose une correction pour chacun, sans rien modifier pour l'instant. »

Réparer

11. Diagnostic SAVE

« Ne modifie rien. Symptôme : [ ]. Action pour reproduire : [ ]. Voulu : [ ]. Erreur exacte : [console]. Changement récent : [ ]. Donne la cause la plus probable, les autres causes possibles et la plus petite correction. »

12. Messages de suivi

« Ajoute des messages dans la console qui affichent les données reçues à chaque étape de [fonction], pour que je voie où le problème apparaît. Je les retirerai ensuite. »

13. Retour arrière

« Annule les dernières modifications et reviens à la version où [ce qui fonctionnait]. Explique-moi ce qui sera perdu. »

Sécuriser et livrer

14. Revue de sécurité

« Fais une revue de sécurité complète, sans rien modifier : règles d'accès par table et par rôle, secrets exposés dans l'interface, vérification des saisies côté serveur, pages accessibles sans connexion, bibliothèques obsolètes ou inconnues, données personnelles collectées. Classe du plus grave au moins grave. »

15. Test des droits

« Écris des tests qui vérifient qu'un utilisateur du rôle [ ] ne peut ni lire ni modifier les données d'un autre utilisateur, et montre-moi les résultats. »

16. Performance et accessibilité

« Analyse la page [ ] pour le téléphone : poids des images, bibliothèques lourdes, contrastes, textes alternatifs, taille des boutons. Propose les améliorations par ordre d'impact. »

17. Page Confidentialité

« Rédige une page Confidentialité claire et courte, en français simple, pour cette application : données collectées, finalités, durée de conservation, destinataires, droits des personnes (accès, rectification, opposition, suppression) et contact. Je la ferai vérifier. »

18. README et guide client

« Rédige un README technique (sans aucune valeur secrète) et un guide d'utilisation d'une page pour le client, avec captures à insérer. »

Relier et automatiser

19. Intégrer un service

« Intègre [service] à partir de sa documentation : [lien]. Les clés sont dans les variables d'environnement [NOMS]. Tout appel se fait dans une fonction serveur. Gère les erreurs et les délais. Présente d'abord le plan et les cas d'échec. »

20. Configurer une automatisation

« Je veux automatiser : quand [déclencheur], faire [actions], seulement si [condition]. Avec [Make ou Zapier], décris-moi pas à pas les modules à ajouter et les champs à relier, et les tests à faire. »

Annexe C · Pour aller plus loin

Les références de ce livre

- **Andrej Karpathy**, message du 2 février 2025 qui a lancé l'expression « vibe coding » (x.com/karpathy).
- **OWASP Top 10** (owasp.org) : la liste de référence des failles de sécurité des applications web.
- **Snyk**, « Is vibe coding secure? » : les risques de sécurité du code généré par l'IA.
- **DigComp 2.2** (Commission européenne) : création de contenu numérique, programmation et résolution de problèmes au niveau avancé.
- **Loi n° 2013-450** relative à la protection des données à caractère personnel ; **loi n° 2013-451** relative à la lutte contre la cybercriminalité ; **loi n° 2013-546** relative aux transactions électroniques ; **ARTCI**.

Les documentations à garder sous la main

- La documentation de **votre** constructeur d'applications (Lovable, Bolt, Replit, Google AI Studio…).
- **docs.github.com** : dépôts, commits, branches, Pull Requests, sécurité.
- **vercel.com/docs** et **docs.netlify.com** : déploiement, variables d'environnement, domaines.
- **supabase.com/docs** : base de données, authentification, sécurité au niveau des lignes (RLS).
- La documentation de votre **agrégateur de paiement** : création de paiement, webhooks, vérification des signatures.
- **help.make.com** et **help.zapier.com** : scénarios, filtres, requêtes HTTP.
- Le site du registre **.ci** (NIC.CI) : liste des bureaux d'enregistrement accrédités.

Et après ?

Le vibe coding évolue chaque mois. Restez curieux, prudent et honnête. Si le métier vous passionne, apprenez les bases de JavaScript et de SQL : vous dirigerez l'IA encore mieux. Et surtout, transmettez : la collection « Apprivoiser l'informatique » est née de la conviction que personne ne doit rester au bord du numérique.

ATTESTATION DE FIN DE PARCOURS

Tome 3 · Créer avec l'IA

Je soussigné(e), formateur ou formatrice, atteste que

.....

a suivi l'ensemble des modules du Tome 3, réalisé les exercices et présenté avec succès le projet final « Une application complète pour un vrai client ».

L'apprenant sait désormais concevoir une application web, la faire construire par l'intelligence artificielle, la vérifier, la corriger, la sauvegarder, la mettre en ligne, la sécuriser, la relier à des services externes, automatiser des tâches et proposer ses services à des clients.

Fait à .....

Le .....

Signature du formateur : .....

Signature de l'apprenant : .....

FOR LIFE